Multi-Label Ambiguity Detection in Software Requirements Using Language Models

by

Paniz Oghabi

A thesis submitted to the Office of Graduate Studies

in partial fulfillment of the requirements for the degree of

Master of Applied Science

in

Data Science, Analytics, and Artificial Intelligence

Carleton University
Ottawa, Ontario

©2025
Paniz Oghabi

ABSTRACT

In software engineering, requirements specification plays an important role in capturing and analyzing stakeholders' needs. However, as software requirements are typically written in natural language, they usually have ambiguity. Identifying and understanding the ambiguity in requirements enables development teams to prevent misunderstandings, address defects earlier, and reduce project costs and failures. While recent studies have focused on detecting only one or two types of ambiguity, this study investigates whether language models (LMs) can detect multiple types of ambiguity, specifically structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness. We collected 1,266 real-world requirements from the public requirements (PURE) dataset, which includes software requirements specifications (SRS) from several domains, and labeled them using previous methods. Two LMs, DistilBERT and RoBERTa, were fine-tuned using the labeled dataset. Their performance was evaluated on a test set of 255 manually annotated requirements. RoBERTa achieved a higher macro-averaged F1-score (75.61%) compared to DistilBERT (73.15%). Applying back-translation (BT) data augmentation to address class imbalance further improved RoBERTa's macro-averaged F1-score to 78.84%. Compared to existing methods, our multi-label model achieved approximately 12% and 7% higher F1-scores in scope ambiguity and structural ambiguity detection, respectively, while demonstrating slightly lower results for anaphoric ambiguity (about 1.8%) and vagueness detection (about 3%).

ACKNOWLEDGEMENTS

I would like to express my gratitude to my supervisor, Dr. Nafiseh Kahani, for her invaluable guidance and support throughout this research. Her mentorship and insightful feedback contributed to the successful completion of this work. I sincerely appreciate all her dedication and am thankful for her time and effort in guiding me through my academic journey.

I am deeply grateful to my family, without whom pursuing my master's degree would not have been possible. Their constant emotional support, patience, and belief in my abilities provided me with the strength and motivation necessary to achieve my academic goals. I cannot thank them enough for always being there for me.

Finally, I would like to extend my thanks to my dear friends, Bahareh Abolhasanzadeh and Sadegh Tabassi, whose assistance and support have been pivotal in my academic pursuits. Their encouragement and friendship have enriched this entire experience.

TABLE OF CONTENTS

Abstract **I**

Acknowledgements **II**

LIST OF ABBREVIATIONS **VI**

List of Figures **VII**

List of Tables **VIII**

1 Introduction **1**

1.1 Research Questions 2
1.2 Key Contributions 3
1.3 Thesis Organization 4

2 Background **5**

2.1 Types of Ambiguity 5
2.1.1 Rationale for Exclusion of Certain Ambiguity Types 6
2.1.2 Scope of Ambiguity Detection 7
2.2 Language Models 7
2.2.1 Transformer Encoder Architecture 9
2.2.2 Pre-training and Fine-tuning 9
2.2.3 Transformer-based Encoder Model 10
2.2.4 Multi-Label Classification of Transformer-based Econder Model 12
2.3 Data Augmentation 12
2.3.1 Evaluation Metrics for Back-Translation 13
2.4 Inter-Rater Agreement 14
2.4.1 Cohen's Kappa 14
2.4.2 Fleiss' Kappa 15
2.5 Summary 15

3 Literature Review **17**

3.1 Rule-based Methods 17
3.1.1 Summary 19

3.2 Machine Learning Methods 20
3.2.1 Summary 25

4 Methodology: Dataset Creation and Model Development 26
4.1 Dataset Creation 26
4.1.1 Data Collection 26
4.1.2 Data Preprocessing 28
4.1.3 Ambiguity Annotation 28
4.2 Model Development 31
4.2.1 Dataset Splits 32
4.2.2 Data Augmentation 32
4.2.3 Model Configuration and Fine-tuning 34
4.3 Summary 37

5 Evaluation 39
5.1 Manual Test Set 39
5.2 Evaluation Metrics 40
5.2.1 Individual Ambiguity Type Performance 40
5.2.2 Multi-Label Performance 41
5.3 Results and Analysis 42
5.3.1 Which fine-tuned language model achieves the highest baseline performance for multi-label ambiguity detection? (RQ1) 42
5.3.2 Which language model provides the best performance after being fine-tuned with BT augmentation? (RQ2) 43
5.3.3 How does our unified multi-label model, the best-performing model from RQ2, compare to previous detection methods? (RQ3) 45
5.4 Summary 50

6 Conclusion and Future Work 51
6.1 Threats to Validity 51
6.2 Future Work 52
6.2.1 Dataset Refinement and Expansion 52
6.2.2 Expanding Ambiguity Type Coverage 53
6.2.3 Integration into Interactive Feedback Systems 53
6.2.4 Hyperparameter Optimization 53

References 55

A Additional Figures **64**

LIST OF ABBREVIATIONS

BT	Back-Translation
LMs	Language Models
ML	Machine Learning
NLP	Natural Language Processing
SRS	Software Requirements Specifications
POS	Part-of-Speech
BLEU	Bilingual Evaluation Understudy
METEOR	Metric for Evaluation of Translation with Explicit ORdering
PURE	Public Requirements
LR	Logistic Regression
KNN	K-Nearest Neighbors
SVM	Support Vector Machine
RF	Random Forest
NB	Naive Bayes
DT	Decision Trees
ULMFiT	Universal Language Model Fine-Tuning
BOW	Bag of Words
TF-IDF	Term Frequency-Inverse Document Frequency
SBADR	Score-Based Ambiguity Detector and Resolver
SREE	Systemized Requirements Engineering Environment
QuARS	Quality Analyzer for Requirement Specifications
QQ	Quantifier–Quantifier
QN	Quantifier–Negation
QA	Quantifier–Adverb
UML	Unified Modeling Language

LIST OF FIGURES

2.1 Encoder-Decoder Architecture 8
2.2 Transformer Encoder Architecture [1] 10

4.1 Overall Methodology for Detecting Ambiguity Types in Software Requirements 27
4.2 Distribution of Requirement Lengths based on Word Count (Training set and Augmented data) 35

5.1 Comparison of RoBERTa Model before and after Data Augmentation with Back-translation 44
5.2 Confusion Matrices for Structural Ambiguity 46
5.3 Confusion Matrices for Scope Ambiguity 47
5.4 Confusion Matrices for Anaphoric Ambiguity 48
5.5 Confusion Matrices for Vagueness 49

A.1 Distribution of Token Lengths in DistilBertTokenizer 64
A.2 Distribution of Token Lengths in RobertaTokenizer 64

LIST OF TABLES

2.1 Ambiguity Classes [2, 3] 7
2.2 Comparison of Language-based Models 11
2.3 Interpretation of Kappa Values 14

3.1 Rule-based Ambiguity Detection Methods in Prior Work 18
3.2 Machine Learning Ambiguity Detection Methods in Prior Work . 23

4.1 Comparison between PURE Dataset Requirements and the pre-processed version Extracted by Luitel et al.[4] 28
4.2 Comparison of Average BLEU Scores for Back-Translation 33
4.3 Key Hyperparameters Used for Fine-tuning 37
4.4 Examples of Multi-Label Ambiguity Classification Output 38

5.1 Fleiss' Kappa Value for Each Type of Ambiguity 39
5.2 Comparison of DistilBERT and RoBERTa in Detecting Ambiguity Types 43
5.3 Comparison of DistilBERT and RoBERTa in Detecting Ambiguity Types after Back-Translation Augmentation 43
5.4 Performance Comparison Between Proposed and Existing Methods for Each Ambiguity Type 45

Chapter 1

Introduction

Software requirements determine what a system or an application requires to meet stakeholders' needs. Therefore, it is crucial that they are expressed in a precise and clear way to make successful software development. The quality of the requirements is significantly related to reducing defects, which occur frequently during the requirements phase compared to different development processes, such as coding phase and designing phase [5].

Several studies have identified different classifications of defects in software requirements specifications (SRS), such as ambiguity, incorrectness, inconsistency, omission, superfluity, and non-conformance to standards [6]. Among these, ambiguity in software requirements is identified as one of the major factors affecting software quality [7]. Ambiguity occurs when a requirement statement can be interpreted in multiple ways, contributing to confusion, errors, and increased development costs and time [8]. During software system implementation, unresolved ambiguity can lead to disagreements between stakeholders and developers, which can potentially cause software failures [9]. A notable real-world example is NASA's Mars mission failures in 1998–1999, where ambiguous interface specifications between software systems contributed to catastrophic losses worth hundreds of millions of dollars [10]. Such cases highlight the critical importance of addressing ambiguity early in the development process, as the cost of corrections and rework increases in later stages of a project [11].

Understanding why ambiguity remains so prevalent requires examining how requirements are documented. Requirements are mostly written in natural language [12], which allows for flexibility and widespread understanding among diverse teams [13]. An online survey conducted by Mich et al. [14] involving 151 software companies found that in 95% of cases, natural language was used to define requirements documents. This widespread use of natural language was further supported by Kassab et al. [15], who conducted a survey with 250 practitioners, finding that a majority (61%) indicated that their organizations typically use natural language to specify software and system requirements. Given this prevalence and the unique challenges associated with natural language ambiguity, this research specifically focuses on detecting different types of ambiguity in natural language software requirements.

Various methods have been developed to detect ambiguity in software requirements. Traditional approaches, such as manual reviews, structured walkthroughs, scenario-based reading [16], and checklists [8], depend on human expertise, limiting their effectiveness for large projects due to the considerable time investments required and the need to create checklists [17]. Current methods for detecting ambiguity in software requirements rely on rule-based [18, 19, 20, 21], and machine learning (ML) methods [22, 23, 24, 25]. However, these methods have some limitations, such as dependency on predefined dictionaries, binary ambiguity clas-

sification without identifying specific ambiguity types [24, 25], or a narrow focus restricted to one or two ambiguity categories. Therefore, there is a critical need for an automated approach capable of detecting different types of ambiguity.

In this research, we present a hybrid approach integrating existing rule-based and ML methods to label four types of ambiguity, consisting of structural ambiguity, anaphoric ambiguity, scope ambiguity, and vagueness. We employ established methods: Score-Based Ambiguity Detector and Resolver (SBADR) [26] for structural ambiguity, TAPHSIR [22] for anaphoric ambiguity, rule-based heuristic [27, 3] for scope ambiguity, and predefined word lists [19, 28] for vagueness detection. Our labeled dataset, generated from these methods, is then used to train two language models (LMs), RoBERTa [29] and DistilBERT [30], for automatic detection of ambiguity types. Evaluating the trained models' performance on a test set dataset of 255 unseen requirements annotated by human experts showed that RoBERTa achieved higher evaluation metrics, including a macro-averaged recall of 88.24%, precision of 67.78%, F1-score of 75.61%, along with a hamming accuracy of 83.17%. After fine-tuning both models with additional data produced through back-translation (BT) augmentation on minority labels within structural ambiguity, scope ambiguity, and anaphoric ambiguity, RoBERTa maintained superior performance across all metrics, outperforming DistilBERT by 5.03% in macro-averaged F1-score. Although our approach surpassed the dedicated methods on scope ambiguity and structural ambiguity detection, achieving F1-scores of 79.52% and 74.22%, respectively, it showed slightly lower performance for anaphoric ambiguity (89.80% vs. 91.67%) and vagueness detection (71.82% vs. 74.77%). These results have also been published in a conference paper [31].

1.1 Research Questions

In this work, we address the following research questions:

RQ1. Which fine-tuned language model achieves the highest baseline performance for multi-label ambiguity detection?

This research question aims to establish a performance baseline by identifying which of the two selected transformer-based models, RoBERTa or DistilBERT, is more effective for multi-label ambiguity detection. These baseline models were chosen to explore the trade-offs between DistilBERT, a smaller and faster model, and RoBERTa, a larger model pre-trained on a more diverse corpus. Both models were fine-tuned on the labeled requirements, and then their performance was evaluated on the test set.

RQ2. Which language model provides the best performance after being fine-tuned with BT augmentation?

This question assesses the impact of our data augmentation strategy, focusing on mitigating the class imbalance observed in our dataset. To achieve this, we fine-tuned both RoBERTa and DistilBERT (augmented models) on an expanded dataset that combined the training data with additional augmented samples. These new samples were generated using BT, targeting the minority labels within structural ambiguity, scope ambiguity, and anaphoric ambiguity. The goal is to determine the overall best-performing model and to examine whether the superior baseline model identified in RQ1 retains its advantage after data augmentation.

RQ3. How does our unified multi-label model, the best-performing

model from RQ2, compare to previous detection methods?
The primary objective of this research question is to determine whether a unified multi-label classification approach performs comparably or better than previous ambiguity detection methods. This evaluation specifically involves taking the best-performing BT augmented model from RQ2 and comparing it against four existing, specialized methods, each designed to detect different types of ambiguity. The analyses were conducted individually across each of the four ambiguity types to assess the advantages and potential trade-offs of employing a single, unified model.

1.2 Key Contributions

The main contributions of this thesis include:

- **Multi-Label Detection:** Unlike previous studies that have focused on detecting only one or two ambiguity types, relied on fixed word lists, or used binary classification that does not mention the specific type, this research presents a unified approach for detecting structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness in software requirements. This approach eliminates the need for using separate detection systems and provides an integrated view of ambiguity patterns in requirements documents. The multi-label classification approach helps software developers identify multiple types of ambiguity in requirements, reducing potential negative impacts on software development.

- **Transformer-based Models:** This study employs two transformer-based LMs (RoBERTa and DistilBERT), specifically fine-tuned for ambiguity detection in software requirements. These models, pre-trained on corpora of different sizes, provide insights into the trade-offs between model complexity and performance in the domain of requirements engineering.

- **Data Augmentation:** To address the class imbalance problem in ambiguity detection datasets, this research demonstrates the effectiveness of BT as a data augmentation technique. The augmentation technique generates synthetic training examples while preserving the linguistic characteristics and ambiguity patterns of the original requirements. This contribution shows measurable performance improvements, particularly for underrepresented ambiguity types, making our approach more practical for real-world applications.

- **Real-World Dataset:** The research utilizes a dataset of 1,266 real-world requirements from the public requirements (PURE) dataset, spanning multiple project domains and representing diverse industrial contexts. This dataset, annotated using established methods from previous research, provides a robust foundation for training models. The diversity of the dataset ensures that the proposed approach can generalize across software requirements in different software projects.

1.3 Thesis Organization

The remainder of this paper is organized into 5 chapters. Chapter 2 provides the fundamental background necessary for this study. It introduces different types of ambiguity in software requirements, key concepts, and theoretical frameworks related to transformer-based LMs, data augmentation techniques, and inter-rater agreement. Chapter 3 reviews existing work on ambiguity detection, covering both rule-based methods and ML methods, and identifying current research gaps. Chapter 4 outlines the research methodology, including data collection, preprocessing, labeling procedures, and model development. Chapter 5 presents an assessment of the experimental results. It includes a manual test set, detailed analyses of model performance metrics, comparisons of different models, and an evaluation of our best-performing model against previous studies. Finally, Chapter 6 summarizes our research conclusions, discusses threats to validity, and suggests directions for future work.

Chapter 2

Background

This chapter provides background knowledge for this thesis. The content is organized under four main topics: types of ambiguity, LMs, data augmentation, and inter-rater agreement.

2.1 Types of Ambiguity

Ambiguity generally occurs when a sentence or a phrase has more than one meaning or interpretation [14]. Riaz et al. [2] categorize ambiguity into the following types:

- **Lexical Ambiguity:** This ambiguity occurs when a single word can be interpreted in different ways [32]. This type divides into *homonymic*, where one word has several unrelated meanings (e.g., "bank" meaning "river bank" and "financial institution"), and *polysemic*, where a word has related meanings sharing a common origin (e.g., "head" as a body part versus a leader of an organization).

- **Structural (syntactical) Ambiguity:** When a sentence has different grammatical structures, and each interpretation has a different meaning, it is considered structurally ambiguous. This type includes different subtypes such as *attachment* ambiguity and *coordination* ambiguity. The examples provided here are taken from the PURE dataset [33]. Attachment ambiguity occurs when phrases can attach to different parts of the sentence. For example, in the requirement "The system shall backup files modified by users with encryption" it is unclear whether the prepositional phrase "with encryption" modifies the verb "backup" or the verb phrase "modified by users". Coordination ambiguity involves multiple conjunctions or one conjunction with a modifier. "All telescope, instrument, and detector control information is to be available at any operation level" is considered an example of coordination ambiguity. The first interpretation is that "control information" applies to "telescope", "instrument", and "detector", while another interpretation is that it applies only to "detector". In addition, Osama et al. [26] identify another subtype *analytical* ambiguity, caused by unclear roles of sentence constituents. For the requirement, "The user shall be able to select pretty-printed text files to display" two parses are possible: "pretty-printed" modifies "text files" (select files that have already been formatted), and "pretty-printed text" forms a compound noun, and the user selects the files containing such text.

- **Semantic Ambiguity:** This type of ambiguity is present when a grammatically correct sentence can be interpreted in multiple ways because the

context is confusing or unclear. Two common subtypes are *scope* ambiguity and *anaphoric* ambiguity. Scope ambiguity occurs when a sentence has more than one logical interpretation in the context [3]; for example, "Each transaction must be approved by a manager" means (i) one designated manager approves all transactions or (ii) every transaction requires approval, but different managers may approve different transactions. Anaphoric ambiguity appears when a pronoun can refer to multiple antecedents, as in "The system processes the data and stores it in the database" where "it" could denote either the system or the data.

- **Pragmatic Ambiguity:** This ambiguity occurs when sentence interpretation depends on contextual factors beyond the text itself [34]. This requires both linguistic elements (surrounding textual context) and extra-linguistic factors (situational circumstances, domain knowledge, and communicative expectations of stakeholders) [35, 36]. Ferrari and Gnesi [36] provide several detailed examples highlighting pragmatic ambiguity in software requirements.

In addition, Berry et al. [3] further recognize vagueness and language errors as additional sources of interpretational problems in requirements specifications. These categories, while distinct from classical ambiguity types, present similar challenges in achieving clear and unambiguous communication between stakeholders:

- **Vagueness:** A requirement has vagueness when it lacks precise measurability or admits borderline cases (e.g., "The system shall provide a fast response time", where "fast" is vague because the acceptable threshold is undefined).

- **Language Errors:** These arise when incorrect grammar, punctuation, and word choice lead to unintended interpretations. For example, in "Users must update his password every 90 days", the pronoun–antecedent mismatch (plural users vs. singular his) makes it unclear whether the rule applies to each individual user or to a single shared account. However, these errors often manifest as other types of ambiguity and lack specific methods for detection.

Table 2.1 summarizes the main ambiguity categories identified in requirements engineering literature. However, not all types are equally suitable for automated detection given current technological capabilities and practical constraints.

2.1.1 Rationale for Exclusion of Certain Ambiguity Types

We exclude certain types of ambiguity, such as lexical ambiguity, pragmatic ambiguity, and language errors, from our detection approach due to the lack of reliable detection methods or the necessity of additional contextual information beyond our work. In lexical ambiguity, though the goal is to find words with multiple meanings, methods using lexical databases like WordNet or BabelNet tend to over-identify ambiguity [37]. Since many words naturally have several dictionary meanings, almost every requirement includes at least one such word.

Table 2.1: Ambiguity Classes [2, 3]

Ambiguity Classes	**Definition**
Lexical Ambiguity	A word can have multiple meanings.
Structural Ambiguity	A sentence can be parsed in more than one grammatical structure, each causing a different meaning.
Semantic Ambiguity	A grammatically correct sentence that conveys multiple interpretations.
Pragmatic Ambiguity	Interpretation depends on external context or domain knowledge.
Vagueness	Specifications are so imprecise that they cannot be measured clearly.
Language Errors	Errors arise from grammar mistakes, incorrect punctuation, or inappropriate word choice.

Pragmatic ambiguity is also removed because it requires context beyond the requirement statement itself. Although Riaz et al. [2] mention referential ambiguity as a common subset of the pragmatic category, it overlaps with anaphoric ambiguity [3], and also, based on existing pragmatic detection methods, this type requires extensive background documentation and project-specific knowledge bases [34, 35]. Since our analysis focuses on individual requirement statements rather than broader project contexts, pragmatic ambiguity detection, which necessitates domain knowledge graphs, remains outside our scope. We also exclude language errors because they lack systematic detection methods and may manifest through other ambiguity categories. Additionally, reliable detection of lexical ambiguity, pragmatic ambiguity, and language errors would require extensive manual annotation and subjective judgments that could compromise the objectivity and reproducibility essential to our automated detection approach.

2.1.2 Scope of Ambiguity Detection

Based on the rationale above, this research targets four key ambiguity types selected for their prevalence and suitability for automated detection. We treat scope ambiguity and anaphoric ambiguity as separate categories rather than grouping them under semantic ambiguity because they require distinct labeling methods and represent different linguistic phenomena, as detailed in 4.1.3. Therefore, our work targets four ambiguity types: *structural ambiguity*, *scope ambiguity*, *vagueness*, and *anaphoric ambiguity*. These categories can be detected using available methods and cover the most important ambiguity problems in requirements.

2.2 Language Models

LMs are computational models that process and interpret human language. These probabilistic deep learning models estimate the likelihood of word sequences, enabling them to capture semantic relationships, syntactic structures, and contextual dependencies within text [38]. This capability allows them to perform

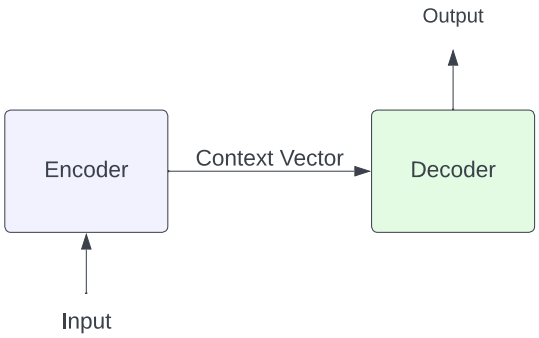


Figure 2.1: Encoder-Decoder Architecture

various natural language processing (NLP) tasks, including text classification, summarization, translation, named entity recognition, and question answering [38]. The development of LMs has evolved from traditional statistical approaches [39] through neural approaches [40, 41] to the current transformer-based architectures that have revolutionized the field since 2017 [38].

The LMs used in this thesis are based on the Transformer architecture [1], specifically utilizing its encoder component for text understanding tasks. Transformers revolutionized NLP by introducing the self-attention mechanism, which allows the model to process sequential data like sentences by dynamically weighing the importance of different words in the sequence relative to each other. The transformer model, introduced by Vaswani et. al. [1], consists of two main parts, as illustrated in Figure 2.1:

- **Encoder:** This component reads and processes the input text sequence to create contextual representations (often called a "context vector") that capture the semantic and syntactic meaning of words within their context. Unlike previous recurrent architectures that process words sequentially, the encoder processes the entire input simultaneously to capture bidirectional context through parallel computation.

- **Decoder:** This part takes the encoder's representation and generates an output sequence. In tasks like machine translation, the decoder generates the translated sentence word by word, using both the encoder's context and previously generated tokens.

Although the complete encoder-decoder architecture is highly effective for generative tasks like machine translation, the architecture has been modified for different applications. For tasks centered on natural language understanding, such as text classification, the decoder is unnecessary. In these cases, a specialized encoder-only architecture is employed, as its purpose is to generate a high-quality, contextual representation of the input text, which is sufficient for making a classification decision.

2.2.1 Transformer Encoder Architecture

The encoder processes input text through several key components that work together to create contextual representations. Since the self-attention mechanism does not inherently capture word order, transformer encoders use positional encoding. Positional encodings are numerical vectors that represent the absolute position of each token in the sequence. These vectors are added element-wise to the input embeddings (the initial numerical representations of words), providing the model with both semantic information about each word and its positional context within the sequence.

Figure 2.2 illustrates the overall architecture shown in the transformer encoder [1]. The prepared input (input embeddings combined with positional encodings) is then processed through a stack of N identical encoder blocks. Each encoder block consists of two main sub-layers connected by residual connections and layer normalization:

- **Multi-Head Attention:** This mechanism allows each token to attend to all other tokens in the sequence, learning which words are most relevant for understanding each token's meaning in context. The "multi-head" aspect means this attention is computed multiple times in parallel, each focusing on different types of relationships. Some heads might capture syntactic dependencies while others focus on semantic relationships. This architecture allows the model to capture diverse linguistic patterns and relationships through parallel processing.

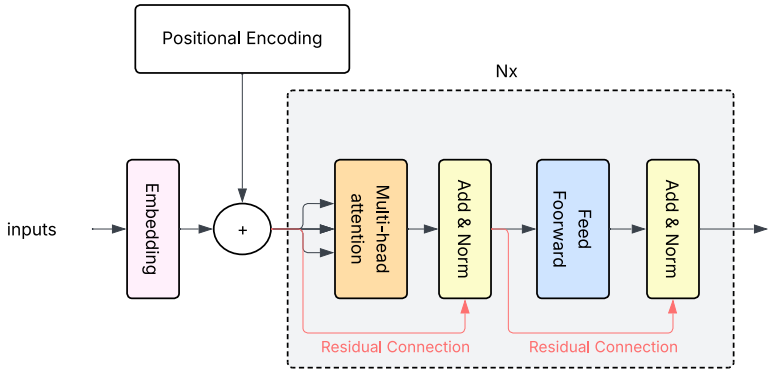
- **Feed-Forward Network:** Following the attention mechanism, each position is processed independently through a fully connected feed-forward network. This network consists of two linear transformations with a GELU activation function. This component adds non-linearity to the model and allows it to learn richer transformations of the attention outputs, essential for learning more complex representations of the data.

Both sub-layers include residual connections and layer normalization (shown as "Add & Norm" in Figure 2.2). Residual connections help with gradient flow during training for deeper construction of networks, while layer normalization stabilizes training and improves convergence. Through this architecture, each encoder layer progressively builds more abstract and contextual representations of the input text. The lower layers typically capture local syntactic patterns, while higher layers encode more global semantic relationships.

2.2.2 Pre-training and Fine-tuning

The effectiveness of Transformer-based LMs largely results from a two-stage training approach: pre-training and fine-tuning.

Pre-training involves training a large model on a massive amount of unlabeled text data, often billions of words, using self-supervised learning tasks. During this stage, the model learns general language understanding by predicting masked words in a sentence (masked language modeling), sentence order prediction, denoising, and sentence prediction [42, 43, 44]. This helps the model have a broad understanding of language that can be applied to various tasks.



Note: "Nx" denotes the repetition of the encoder block N times; black arrows show the main data flow; red arrows represent residual connections; "+" denotes element-wise addition operations.

Figure 2.2: Transformer Encoder Architecture [1]

In the fine-tuning stage, the pre-trained model is adapted to a specific task using a smaller, labeled dataset. For instance, for sentiment analysis, the model is further trained on sentences labeled as positive, negative, or neutral. During fine-tuning, the pre-trained model weights provide the starting point, and the model adapts to learn patterns specific to the target task while retaining the general language understanding acquired during pre-training [38].

This two-stage approach provides significant advantages. Models can leverage vast amounts of unlabeled text during pre-training, while requiring relatively small amounts of task-specific labeled data for fine-tuning. For specialized domains like software requirements, this is particularly valuable since obtaining large amounts of expertly labeled ambiguity data is expensive and time-consuming, whereas the general language understanding from pre-training provides a strong foundation for recognizing linguistic patterns relevant to ambiguity detection.

2.2.3 Transformer-based Encoder Model

This research focuses on encoder-based models because the research objective is centered on text understanding for classification, not text generation. Since the goal is to classify software requirements for different types of ambiguity, the encoder's ability to create rich contextual representations makes it the most efficient architecture for the task.

BERT

BERT (Bidirectional Encoder Representations from Transformers) [44] introduced a paradigm shift in NLP by being the first transformer encoder model to achieve truly bidirectional context understanding. BERT simultaneously considers context from both directions, enabling deeper semantic understanding.

BERT employs two pre-training objectives. First, masked language modeling randomly masks approximately 15% of input tokens, training the model to predict them based on bidirectional context. Second, next sentence prediction teaches the

model to understand relationships between sentence pairs by predicting whether one sentence logically follows another. BERT-based architecture consists of 12 transformer encoder layers with 110 million parameters and was pre-trained on 16GB of data.

RoBERTa

RoBERTa (Robustly Optimized BERT Pre-training Approach) [29] is an improved version of BERT that addresses several limitations of the original model. The creators of RoBERTa did not change the fundamental architecture of BERT. Instead, they optimized its pre-training process.

The primary optimizations include: (i) removing the next sentence prediction objective, as empirical analysis showed it provided little benefit for most downstream tasks, (ii) implementing dynamic masking instead of static masking, where masked token patterns change across training epochs rather than being fixed during data preprocessing, (iii) training on larger datasets (160GB vs. 16GB for BERT) with longer sequences, and (iv) using larger batch sizes with optimized learning rate schedules. RoBERTa-based model architecture includes 12 encoder layers and 125 million parameters. These methodological improvements result in more robust language representations without increasing model complexity.

DistilBERT

DistilBERT [30] is a compressed version of BERT that maintains much of BERT's performance while being significantly smaller and faster. The distillation process involves training DistilBERT to reproduce the output probabilities of BERT rather than just the final predictions. It retains approximately 60% of BERT's parameters while achieving about 97% of its performance on various tasks. The model uses the same transformer architecture as BERT but with fewer layers, making it more computationally efficient. Its architecture includes 6 encoder layers with 66 million parameters. This efficiency makes DistilBERT particularly attractive for applications where computational resources are limited or where fast inference is required. Despite its smaller size, DistilBERT maintains the bidirectional understanding capabilities that make BERT effective for language understanding tasks.

Table 2.2: Comparison of Language-based Models

Model	**Layers/Parameters**	**Training Data**	**Key Innovation**
BERT	12/110M	16GB	Bidirectional context
RoBERTa	12/125M	160GB	Optimized training procedure with dynamic masking and more data
DistilBERT	6/66M	Distilled from BERT	Knowledge distillation to compress BERT

Note: "M" indicates million parameters.

Table 2.2 summarizes the key characteristics, training data sizes, and the number of layers and parameters for the three models. Building on BERT as the foundational model, this thesis focuses on RoBERTa and DistilBERT to analyze the performance-efficiency trade-off. RoBERTa represents the high-performance option with its robust pre-training on a larger corpus, while DistilBERT is included as a computationally efficient alternative that is significantly smaller and faster.

2.2.4 Multi-Label Classification of Transformer-based Econder Model

To adapt an encoder model for a multi-label classification task, a task-specific "head" is added on top of the pre-trained transformer base. This head is typically a single linear layer where the number of output neurons corresponds to the total number of possible labels. Unlike in multi-class classification, which uses a Softmax activation to select a single best label, multi-label classification employs a Sigmoid activation function on each output neuron independently. The formula for the Sigmoid function is:

$$\sigma(z_i) = \frac{1}{1 + e^{-z_i}}, \tag{2.1}$$

where z_i is the raw output score (logit) for the i-th label.

This allows the model to output a probability score between 0 and 1 for each potential label, indicating the likelihood of its presence. Consequently, the model can assign multiple labels simultaneously to a single input text.

2.3 Data Augmentation

Data augmentation expands training datasets by applying transformations to existing samples or generating synthetic data that preserves their original characteristics. This approach was first employed in computer vision [45] through rotation, cropping, shifting, and noise addition [46] and then successfully adapted to NLP to address limited training data and improve model generalization [47]. In NLP, augmentation methods are categorized into three main groups [48]:

- Paraphrasing (using thesauruses, semantic embeddings, LMs, rules, and machine translation methods such as BT and unidirectional translation),
- Noising (including swapping, deletion, insertion, and substitution),
- Sampling (involving rules-based, non-pretrained, pre-trained, self-training, and mixup techniques).

BT has demonstrated particular effectiveness in generating diverse samples while preserving semantic content. This technique is particularly suitable for requirements that may have ambiguity, as it effectively preserves the necessary linguistic patterns essential for classification objectives [25].

2.3.1 Evaluation Metrics for Back-Translation

The quality of sentences generated through BT is commonly assessed using metrics such as Bilingual Evaluation Understudy (BLEU) [49] and Metric for Evaluation of Translation with Explicit ORdering (METEOR) [50].

BLEU

BLEU [49] score measures the similarity between machine-translated text and original text by calculating the overlap of n-grams (contiguous sequences of n words) between the candidate and reference texts. The metric computes precision scores for different n-gram lengths (1-grams through 4-grams) and combines them using a geometric mean. The BLEU score is calculated using the following formula:

$$\text{BLEU} = BP \exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \tag{2.2}$$

where BP penalizes translations that are shorter than the reference, w_n represents the weights for different n-gram precisions (typically uniform weights of 0.25 for 1-4 grams), and p_n is the precision of n-grams for n =1, 2, 3, 4.

The BLEU score ranges from 0 to 1, where higher scores indicate better translation quality. In our work, we adopt the standard of computing BLEU up to n=4 to assess both adequacy and fluency of translations. Unigrams (n=1) check for the presence of correct individual words, while higher order n-grams (n=2, 3, and 4) measure the correctness of phrases and the preservation of grammatical and structural patterns in the target language.

METEOR

METEOR score [50] addresses some limitations of the BLEU score by incorporating both precision and recall, considering synonyms and paraphrases, and accounting for word order. Unlike the BLEU score, which focuses solely on precision, the METEOR score computes the harmonic mean of precision and recall based on unigram matches between the candidate and reference texts. The METEOR score is calculated through the following steps:

- Alignment: Words in the candidate translation are aligned with words in the reference translation based on exact matches, stem matches, and synonym matches.
- Precision and Recall: Calculate precision (P) and recall (R) based on the number of aligned unigrams.
- Harmonic Mean: Compute the harmonic mean:

$$F_{\text{mean}} = \frac{10PR}{R + 9P}. \tag{2.3}$$

- Penalty: Apply a penalty for fragmentation to account for word order differences:

$$\text{METEOR} = F_{\text{mean}}(1 - \text{Penalty}), \tag{2.4}$$

Where α is a parameter that controls the relative importance of precision versus recall.

Using the BLEU score and METEOR score together provides a balanced view by assessing the following aspects:

- Semantic Preservation: How well the translation maintains the original meaning and content.
- Structural Integrity: Preservation of grammatical structure and word order.
- Lexical Similarity: The degree of vocabulary overlap between the original requirement and each back-translated version.

2.4 Inter-Rater Agreement

LMs require datasets with high-quality labels to evaluate their performance effectively. These labels are typically assigned by human annotators, also known as raters. Assuring consistency among different raters is essential for the reliability of the dataset. Inter-rater agreement metrics quantify this consistency, with Cohen's kappa [51] and Fleiss' kappa [52] being two widely used methods.

2.4.1 Cohen's Kappa

Cohen's kappa [51] is a statistical measure used to evaluate inter-rater reliability between exactly two annotators. It measures the agreement beyond what would be expected by chance. Cohen's kappa κ is calculated as:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \tag{2.5}$$

where p_o is the observed proportion of agreement between annotators, and p_e is the expected proportion of agreement occurring by chance.

The interpretation of Cohen's kappa values follows widely accepted guidelines proposed by Landis and Koch [53], as shown in Table 2.3. A κ lower than 0.00 indicates poor agreement, and an increasing κ value reflects better agreement. κ values that indicate moderate agreement or substantial agreement are generally considered acceptable.

Table 2.3: Interpretation of Kappa Values

Kappa Value (κ)	**Interpretation**
$\kappa < 0.00$	Poor agreement
$0.00 \leq \kappa \leq 0.20$	Slight agreement
$0.21 \leq \kappa \leq 0.40$	Fair agreement
$0.41 \leq \kappa \leq 0.60$	Moderate agreement
$0.61 \leq \kappa \leq 0.80$	Substantial agreement
$0.81 \leq \kappa \leq 1.00$	Almost perfect agreement

Cohen's kappa is valuable in NLP, ML, and language model tasks because it provides a standardized measure that accounts for the base rates of different categories. This is especially important when dealing with imbalanced datasets where some categories are more frequent than others. However, Cohen's kappa is limited to measuring agreement between exactly two raters, which restricts its usability in scenarios involving multiple annotators.

2.4.2 Fleiss' Kappa

Fleiss' kappa [52] generalizes Cohen's kappa to evaluate inter-rater reliability among more than two annotators, allowing the measurement of agreement consistency across multiple raters simultaneously. The calculation of Fleiss' kappa follows a similar conceptual framework to Cohen's kappa but accommodates multiple raters. Fleiss' kappa κ_F is defined as:

$$\kappa_F = \frac{\overline{P} - \overline{P}_e}{1 - \overline{P}_e}, \tag{2.6}$$

where $\bar{P}$ is the average agreement among annotators. It is shown as:

$$\overline{P} = \frac{1}{n} \sum_{j=1}^{n} P_j, \tag{2.7}$$

and $\bar{P}_e$ is the average agreement expected by chance. It is calculated as in equation 2.8:

$$\overline{P}_e = \sum_{k=1}^{K} p_k^2, \tag{2.8}$$

where p_k represents the proportion of all assignments to category k.

The interpretation scale for Fleiss' kappa follows the same guidelines as Cohen's kappa, using the Landis and Koch criteria [53]. However, the calculation process is more complex due to the need to account for varying numbers of raters per item and the aggregation of agreement across multiple rater pairs. Fleiss' kappa has several advantages over Cohen's kappa in multi-rater scenarios. It provides a single measure of agreement that summarizes the consistency across all raters, rather than requiring separate calculations for each pair of raters.

Both Cohen's kappa and Fleiss' kappa serve as essential metrics for validating the quality of annotated datasets. These measures confirm that a test set used to evaluate LMs is sufficiently reliable and consistent across different human annotators, where different annotators may have varying interpretations of the same requirement.

2.5 Summary

This chapter establishes the theoretical foundation for the thesis by examining four key areas essential to understanding multi-label ambiguity detection in software requirements. The chapter begins with a taxonomy of ambiguity types, identifying six main categories and establishing the rationale to focus on four specific types, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and

vagueness, based on their prevalence in software requirements and suitability for automated detection. It then provides an in-depth examination of transformer-based encoder architectures, explaining the fundamental components, including multi-head attention mechanisms and feed-forward networks, along with the two-stage training approach of pre-training and fine-tuning. Three encoder models, BERT, RoBERTa, and DistilBERT, are analyzed, and how multi-label classification is applied to encoder models is explained.

The chapter further explores data augmentation techniques, with particular emphasis on BT as an effective method for generating diverse training samples while preserving semantic content and ambiguity patterns. Evaluation metrics for BT quality, including BLEU score and METEOR score, are explained to assess semantic preservation and structural integrity. Finally, the chapter addresses inter-rater agreement measures, explaining Cohen's kappa and Fleiss' kappa for ensuring dataset quality and validating the reliability of human annotations. The subsequent chapter reviews existing methods for ambiguity detection, highlighting their approaches to identifying ambiguity and discussing their limitations within SRS.

Chapter 3

Literature Review

Automated ambiguity detection in software requirements has gained significant attention as organizations seek to improve requirement clarity while reducing manual effort. Several methods have been developed for ambiguity detection. Some approaches focus on binary classification (ambiguous vs. unambiguous), while others focus on one or two ambiguity types, or a few more, using fixed dictionaries. This section reviews key contributions in existing automated ambiguity detection using rule-based and ML methods, examining their methodologies in this domain and their limitations.

3.1 Rule-based Methods

Zait and Zarour [37] propose a technique for detecting and resolving lexical ambiguity and semantic ambiguity (specifically, scope ambiguity) in raw natural language requirements using NLP and semantic web techniques. For lexical ambiguity, they employ parts-of-speech (POS) tagging and BabelNet for detection, and use n-gram context analysis combined with a graph-based centrality algorithm for lexical disambiguation. To detect semantic ambiguity, they provide a list called "blacklist" to identify this type of ambiguity (e.g., flagging words like "few," "any," "several"); however, they do not clearly explain how semantic ambiguity is resolved. Their evaluation is limited to a single case study (RALIC project), and their technique primarily focuses on lexical ambiguity and semantic ambiguity, without fully detecting these types. Their use of BabelNet often leads to categorizing nearly all requirements as lexically ambiguous, and the provided blacklist may not effectively cover all requirements exhibiting scope ambiguity.

Sabriye and Zainon [54] suggest a framework for automatically detecting ambiguity in SRS using POS tagging techniques. The framework focuses on identifying structural ambiguity and syntax ambiguity in natural language SRS documents through a three-component framework: preprocessing, processing, and post-processing. The framework utilizes the Stanford POS tagger to analyze sentence structures and applies rule-based detection methods to identify ambiguous patterns. Their detection rules specifically target syntax ambiguity (sentences without proper punctuation or containing passive voice) and structural ambiguity (containing logical operators like AND/OR). They develop a prototype tool that color-codes detected ambiguities and provides a graphical representation of ambiguity percentages. While the framework provides a systematic approach using established NLP techniques, it is limited to basic rule-based detection methods and lacks detailed evaluation analysis.

Osama et al. [26] present SBADR, an automatic technique for detecting and resolving structural ambiguity in software requirements. SBADR generates k-best parse trees for each sentence with Stanford CoreNLP [55] and then applies

four filtering pipelines to eliminate meaningless interpretations. If more than one tree remains, the tool flags the sentence as ambiguous and shows the bracketed alternatives to the analyst. SBADR achieved 99% recall and 77% F1-score on 126 requirements and reached 100% recall on coordination ambiguity and 90% on attachment ambiguity in complex scenarios. However, the technique is limited to just structural ambiguity detection.

Table 3.1: Rule-based Ambiguity Detection Methods in Prior Work

Reference	**Ambiguity Types**	**Technique**	**Main Limitations**
[37]*	Lexical; scope	POS-tagging + BabelNet; "blacklist" terms	Single case study evaluation; over-detection for lexical; inadequate coverage for scope
[54]	Structural; syntax	Stanford POS + Rule-based patterns	Basic rule-based patterns; lack of evaluation analysis
[26]*	Structural	k-best parse trees + four filtering pipelines	Single ambiguity type focus
[19]	Vagueness (Defect detection)	Dictionary	Limited to static lists; lack of evaluation analysis for each type
[28]	10 categories, including vague terms and coordination	Dictionary	Limited to static lists; lack of evaluation analysis for each type
[18]*	Lexical; referential; coordination; scope; vague	Dictionary	Limited to static lists; lack of evaluation analysis for each type; missing expert comparison
[56]	Coordination; Prepositional-phrase attachment	Pattern matching + corpus heuristics	Unavailable tool (complex setup); specific ambiguity types (a subset of structural ambiguity)
[57]	referential; coordination; scope; vagueness	Thai tokeniser + Dictionary + POS	Performance issue (low recall); classification disagreement; insufficient evaluation analysis by ambiguity type

Note: * indicates work that also supports ambiguity resolution.

Lami et al. [19] develop Quality Analyzer for Requirement Specifications (QuARS), an automatic tool for detecting various linguistic defects in requirements, including vagueness, subjectivity, optionality, implicity, weakness, underspecification, and multiplicity based on static lists. The tool aims to achieve software quality by performing a systematic analysis of requirements specifications to identify defects that could contribute to problems in the development phases. Tjong et al. [28] present Systemized Requirements Engineering Environment (SREE), a prototype ambiguity and defect finder that achieved 100% recall but lower precision. SREE takes a lexical analysis approach through 10 different categories, such as continuance, coordinator, directive, incomplete, weak, vague

terms, quantifier, pronoun, plural, and optional. The key limitations of both SREE and QuARS tools are their reliance on predefined keyword lists and lack of performance evaluation for each class.

Osama and Aref [18] develop an automated tool, Detecting and Resolving Ambiguity, for identifying and resolving ambiguities in natural language SRS. Their approach employs a fixed dictionary-based detection method that targets five types of ambiguity, including lexical ambiguity, referential ambiguity, coordination ambiguity, scope ambiguity, and vague ambiguity. The system uses predefined dictionaries containing ambiguity indicators for each type, followed by a rule-based resolution module with 14 rules to rewrite ambiguous sentences automatically. Evaluation on 36 real software requirement documents showed the tool detected ambiguities in approximately 60% of case studies and achieved a 67% automatic resolution rate. However, the tool relies on static dictionaries, and the evaluation lacks validation of detection and resolution quality for each type and comparison with human expert analysis.

Ezzini et al. [56] create a rule-based tool for detecting coordination ambiguity and prepositional-phrase attachment ambiguity in requirements using domain-specific corpora automatically generated from Wikipedia. Their method combines structural pattern matching with corpus-driven heuristics and achieved 89% recall on 20 industrial requirements documents across seven domains. However, the method is limited to specific ambiguity types (a subset of structural ambiguity) and suffers from implementation complexity and tool unavailability, making practical adoption challenging.

Intana et al. [57] propose a rule-based approach that combines POS tagging with ambiguous words identified by experts to detect ambiguity in Thai SRS. Their method integrates tokenization using PyThaiNLP and POS tagging with the ORCHID corpus, following a three-step framework: tokenization, POS tagging, and detection. The approach first identifies potentially ambiguous words and then classifies them into four categories, containing referential ambiguity, coordination ambiguity, scope ambiguity, and vagueness, based on their POS tags. They developed a prototype tool called Software Requirement Ambiguity Detector and achieved 85.61% precision and 63.48% recall when evaluated on 250 requirements from seven system domains. This work demonstrates the importance of language-specific approaches for ambiguity detection, as Thai presents unique linguistic challenges, including a lack of spaces between words, the absence of punctuation, and no verb tenses. However, the approach has limitations, including low recall performance, disagreement between expert annotations and tool classifications for certain word types, and the evaluation metrics are reported only at the overall level without breakdown by ambiguity categories, making it difficult to assess performance for individual ambiguity types.

3.1.1 Summary

Table 3.1 summarizes different rule-based methods and the limitations of the previous research on ambiguity detection. These key limitations restrict effective ambiguity detection in software requirements. They rely on predefined keyword lists [19, 28, 18, 37], which produce high false positive rates. Moreover, several methods that detect multiple types of ambiguity lack thorough evaluation

analyses [54, 57, 18], while more advanced techniques target only one or two ambiguity types [26, 37, 56]. Therefore, this research investigates whether LMs, specifically transformer-based models, are capable of detecting multiple ambiguity types within a unified approach.

3.2 Machine Learning Methods

Ezzini et al. [23] present a multi-solution study for automated handling of anaphoric ambiguity in requirements. Their work addresses both the detection of anaphoric ambiguity and anaphora resolution by evaluating different approaches, spanning traditional ML techniques, state-of-the-art LMs (SpanBERT), and existing NLP resolvers. The study employs both supervised ML classifiers trained on language features and feature embeddings for detection, as well as fine-tuned SpanBERT models adapted for resolving anaphoric ambiguity in requirements engineering tasks. Their evaluation on industrial datasets containing approximately 1,350 requirements from eight different application domains showed that a supervised ML ensemble achieved the best results for ambiguity detection, while SpanBERT yielded superior performance for anaphora resolution. They subsequently developed TAPHSIR [22], a tool that employs a hybrid approach to detect and resolve anaphoric ambiguity in natural language requirements. The tool addresses anaphoric ambiguity, an underexplored but prevalent ambiguity type that affects up to 20% of industrial requirements, achieving perfect recall (100%) with approximately 96% accuracy in automated resolution. Notably, the tool successfully identified 87% of cases in which annotators disagreed, even though each annotator considered their interpretation clear. This highlights TAPHSIR's effectiveness in detecting subtle ambiguities that manual inspections might miss. Although their method performed effectively in detecting anaphoric ambiguity, the study focuses exclusively on one type of ambiguity.

Yang et al. [58] present an approach to detecting anaphoric ambiguity in natural language requirements. They develop an ML technique using 17 heuristic features spanning linguistic properties (syntactic and semantic constraints), contextual information, and corpus-based statistics. Through human judgment collection from 38 computing professionals on 200 anaphoric instances, they trained classifiers with NB that achieved high recall (about 98%) with precision ranging from 56-82% depending on the ambiguity threshold. Their methodology of using human validation to define what constitutes problematic ambiguity, combined with ML techniques for automatic detection, demonstrates the effectiveness of human-centered approaches for identifying linguistically complex ambiguities in requirements engineering contexts. However, the approach is limited to anaphoric ambiguity detection and depends on setting an ambiguity threshold, which may lead to misclassification of ambiguities if the threshold is not selected appropriately. Moreover, reliance on a generic corpus rather than domain-specific corpora may limit performance in specialized requirements domains. The training dataset consists exclusively of samples containing anaphoric ambiguity, lacking negative examples of non-ambiguous pronoun usage, which limits the system's ability to distinguish ambiguous from unambiguous anaphoric cases in real-world deployment.

Subedi et al. [25] investigate transfer learning with data augmentation for

binary ambiguity detection. They pre-train a Universal Language Model Fine-Tuning (ULMFiT) language model on 1,460 software requirement sentences from 16 industrial and academic projects, then train a classifier on a manually labeled subset of 483 requirements (241 ambiguous, 242 unambiguous) out of 1,460. The authors increase the labeled set by 25% via Spanish BT augmentation, as it gave the highest mean BLEU score of 43.9% among ten languages. After training different models on both manually labeled requirements and augmented ones, the ULMFiT model surpasses classical ML models, including Support Vector Machine (SVM), Logistic Regression (LR), and multinomial Naive Bayes (NB) classifiers. Therefore, the fine-tuned model demonstrates that BT can boost performance when training data is limited. However, the approach only detects ambiguity as binary and does not identify the specific ambiguity types. Furthermore, the manual labeling was performed by a single domain expert without inter-rater reliability validation, which may introduce labeling bias and affect the generalizability of the results.

Izhar et al.[24] propose a hybrid framework combining rule-based techniques with ML models for ambiguity detection. Their method first applies expert-defined rules to flag ambiguous phrases, then uses term frequency-inverse document frequency (TF-IDF) features with ML classifiers, such as Multi-layer perceptron, Random Forest (RF), K-Nearest Neighbors (KNN), Decision Trees (DT), SVM, NB, LR, to improve detection performance. They evaluated on a dataset of 1,553 requirements across 17 project domains (with 1,371 non-ambiguous and only 185 ambiguous requirements), achieving 97% accuracy and 92% F1-score with RF, significantly outperforming traditional rule-based tools and other ML models. Additionally, their analysis revealed a weak correlation between sentiment and ambiguity ($r = 0.22$), suggesting that sentiment analysis may not be effective for ambiguity detection. Despite these strengths, the study has limitations. The complete list of ambiguous terms and rules was not publicly available, the dataset remains imbalanced, and the method does not differentiate between various ambiguity types.

Alharbi [59] investigates an approach for detecting ambiguity in software requirements using different ML models and a fine-tuned transformation model. The study focuses on linguistic rules to label data for training different models. Alharbi applies two main approaches: feature extraction using hidden states from pre-trained models to train traditional classifiers (LR, KNN, SVM), and end-to-end fine-tuning of a DistilBERT model. The fine-tuned transformer approach demonstrated superior performance with a 77% F1-score, representing a 13% improvement over the ML models. The dataset consisted of 600 software requirement sentences from the PURE database, with manual labeling validated through Cohen's kappa coefficient [51], showing substantial inter-rater agreement. While this work demonstrates the effectiveness of transformer models in detecting ambiguity compared to traditional ML models such as LR, SVM, and KNN, it does not detect different ambiguity types.

Osman and Zaharin [60] propose an automated approach for detecting ambiguous SRS using text mining and ML techniques. Their work focuses on Malaysian industrial software requirements written in Malay, addressing the unique challenges of processing requirements in a language with different linguistic characteristics from English. The authors employ nine text classification algorithms,

including One Rule, NB, LR, KNN, Decision Table, J48 (DT algorithm), RF, Decision Stump (single-level decision tree), and Random Tree (single decision tree with random feature selection), to classify requirements as ambiguous or unambiguous. Their approach involves feature extraction using 405 feature-words derived from 180 requirement specifications, with RF achieving the best performance in terms of accuracy and F1-score. The work demonstrates the importance of language-specific approaches for requirement analysis, as it addresses challenges specific to Malay text processing while providing a systematic methodology for ambiguity detection that could be adapted to other natural languages with similar linguistic characteristics. Nevertheless, this approach employs binary classification without ambiguity type differentiation, is restricted to Malay language requirements, and has potential bias from using the same experts for both the manual training set and the test set.

Oo et al. [61] analyse different methods for identifying ambiguity in SRS documents. Some of them include manual approaches (inspection and review techniques), semi-automatic approaches using NLP (rule-based), and semi-automatic approaches using traditional ML techniques. They provide a broad comparison of these methods through a literature review rather than an empirical evaluation. Through their comparative analysis of existing studies, they conclude that among the ML approaches, the NB text classification technique obtained superior performance compared to other ML techniques, such as DT and SVM, in terms of consistency and accuracy for ambiguity detection tasks. However, they acknowledge that these conclusions are based on reviewing previous work rather than conducting new experiments, and note limitations such as the potential for misleading results in certain contexts and the continued need for human expertise in the ambiguity detection process.

Zhang and Ma [62] propose a supervised ML model for automated detection of ambiguity in building requirements to support automated compliance checking. They treat ambiguity detection as a binary text classification problem and test with four different feature representation methods, such as bag of words (BOW), bag of bi-grams, bag of tri-grams, and TF-IDF using SVM. Their dataset consists of 464 sentences from Health Building Notes, with 237 ambiguous and 227 unambiguous sentences manually labeled by domain experts. The best-performing model achieved recall, precision, and accuracy scores of 99.0%, 71.1%, and 78.2%, respectively, with the tri-grams model performing best on recall and the TF-IDF model achieving the highest accuracy (83.0%). However, the study has several limitations, including a relatively small dataset size (464 sentences) with only one ML model (SVM).

Vieira [63] conducts a comparative analysis of ML techniques for classifying requirements ambiguity in software requirements. The study evaluates combinations of two vectorization techniques (BOW and TF-IDF) with four supervised learning algorithms (KNN, SVM, LR, and multinomial NB). The evaluation uses the PROMISE dataset, containing 985 manually labeled requirements classified as ambiguous or unambiguous. The dataset shows class imbalance with 89% unambiguous and 11% ambiguous requirements, requiring the use of the SMOTE oversampling technique. Results indicated that LR achieved the best overall performance with 91% accuracy and macro-averaged F1-score of 74%, followed by SVM and multinomial NB, while KNN performed poorly with only 51% ac-

Table 3.2: Machine Learning Ambiguity Detection Methods in Prior Work

Reference	**Ambiguity Types**	**Technique**	**Main Limitations**
[23, 22]*	Anaphoric	Feature-based ML	Single ambiguity type focus
[58]	Anaphoric	17 heuristics + NB	Single ambiguity type focus; dependent on threshold selection; corpus-dependency; no negative training examples
[25]	Binary	SVM, LR, NB, ULMFiT + BT	Limited to binary classification; potential bias
[24]	Binary	Rule-based + Multi-layer perceptron, RF, KNN, DT, SVM, NB, LR	Unavailable rule lists; imbalanced data; limited to binary classification
[59]	Binary	Rule-based + LR, KNN, SVM, Fine-tuned DistilBERT	Limited to binary classification
[60]	Binary	Dictionary + Nine classifiers (Random Tree, Decision Stump, Decision Table, One Rule, NB, LR, KNN, J48, and RF); RF best	Limited to binary classification; language specific (Malay); potential bias
[61]	Binary (Survey)	Rule-based, manual approaches, ML	No new experiments; qualitative only; missing expert comparison
[62]	Binary	bi-grams, TF-IDF, BOW, bag of trigrams + SVM	Limited to binary classification; small dataset; only SVM tested
[63]	Binary	BoW,TF–IDF + LR, SVM, NB	Single-researcher annotation bias; missing quality evaluation of generated samples; limited to binary classification
[64]	Structural	Rule-based + RF, SVM, K-NN	Overlap across ambiguity types
[65]	Coordination	5 heuristics + LogitBoost	Small dataset; single ambiguity type focus
[66]	No subtype specified	GPT-4 (zero-shot)	High false positives

Note: * indicates work that also supports ambiguity resolution.

curacy. Although LR showed the most balanced F1-score performance across both classes, the study acknowledged that the achieved metrics would require further refinement for practical market implementation. Key limitations include potential labeling subjectivity from single-researcher annotation and binary classification that treats all ambiguity as a single category. Moreover, ambiguity in requirements often depends on semantic and syntactic relationships, not just word frequencies, meaning simple vectorization techniques (BOW, TF-IDF) may miss these crucial linguistic complexities. The study also does not evaluate the quality of SMOTE-generated synthetic samples or assess whether the oversampling technique preserves the semantic characteristics that define ambiguous requirements.

Oo [64] analyzes the performance of text classification algorithms, including SVM, RF, and KNN, to identify structural ambiguity in software requirements. The study develops six linguistic rules to annotate a dataset of software requirements containing structural ambiguity, and then uses them to train ML models. The results showed that RF achieved the highest accuracy (89%), followed by SVM (86%) and KNN (82%). Although all three algorithms performed well, Oo concluded that RF provided the best performance for detecting structural ambiguity and recommended its use in software requirements engineering. However, some rules used in this study have overlap with rules for other ambiguity types and are not exclusively specific to structural ambiguity.

Yang et al. [65] develop an automated approach for detecting nocuous coordination ambiguity, which is a subset of structural ambiguity in natural language requirements, distinguishing between harmful ambiguity that causes misunderstandings and innocuous ones that are typically interpreted by different readers. Their Nocuous Ambiguity Identification tool focuses specifically on coordination ambiguity, employing an ML classification system that uses five heuristics, including coordination matching, distributional similarity, collocation frequency (both local and corpus-based), morphology, and semantic similarity derived from WordNet. The approach first extracts coordination construction patterns from requirements text, then applies named entity recognition techniques to identify coordination constituents, and finally uses a LogitBoost classifier to determine whether detected ambiguities are nocuous or innocuous based on human judgment training data. The LogitBoost algorithm is selected because it outperforms other traditional ML models, including DT, NB, SVM, and LR. Evaluated on a dataset of 138 coordination instances from requirements documents with judgments from 17 computing professionals, the tool achieved up to 75% precision and demonstrated particular effectiveness at ambiguity thresholds between 45% and 75%. The approach is limited to coordination ambiguity as a subset of structural ambiguity and relies on a small evaluation dataset of only 138 instances.

Mahbub et al. [66] investigate GPT-4's effectiveness in detecting ambiguities, inconsistencies, and incompleteness in SRS using an industrial mechanical lung ventilator system. Their zero-shot analysis of a 52-page industrial SRS document revealed that GPT-4 achieved the highest precision in detecting completeness issues (0.61), followed by inconsistencies (0.43) and ambiguities (0.39). The study identified key limitations of large language models in requirements analysis, leading to numerous false positives. Moreover, while large language models demonstrated potential for automating early defect detection, they cannot fully replace human analysts due to their limited domain knowledge in zero-shot scenarios.

3.2.1 Summary

Table 3.2 presents a comparison of existing ML methods and highlights the key limitations identified in prior ambiguity detection studies. Most approaches focus on binary classification and do not recognize the types of ambiguity [25, 24, 59, 62, 63, 66]. While some of these approaches use traditional ML models like SVM and RF, others have begun to adopt modern LMs like DistilBERT and ULMFiT for this binary task [25, 59]. A few papers target a single specific ambiguity type, such as anaphoric ambiguity [23, 22] or structural ambiguity [64]. Additionally, several studies are limited by small datasets, lack a clear explanation of ambiguity labeling criteria before model training, and may suffer from potential bias from single expert annotation without inter-rater reliability validation [24, 62].

Nevertheless, none of these approaches investigates the effectiveness of state-of-the-art LMs in classifying requirements in different types of ambiguity. This thesis addresses this gap by investigating the effectiveness of fine-tuned transformer models for multi-label classification of four specific ambiguity types, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness. The following section outlines the methodology, covering data collection, preprocessing, labeling, and model tuning processes.

Chapter 4

Methodology: Dataset Creation and Model Development

The goal of this study is to determine whether LMs can detect different types of ambiguity in software requirements. Figure 4.1 illustrates the overall pipeline, divided into two major phases: data creation and model development. In the first phase, (1) we collected software requirements in different domains, (2) preprocessed the data, and (3) labeled them for different ambiguity types using established approaches from previous studies. In the model development phase, (4) we split the labeled dataset into a training set and a validation set, and (5) applied data augmentation to address class imbalance within the structural ambiguity, scope ambiguity, and anaphoric ambiguity. Finally, (6) we chose two transformer-based models, DistilBERT [30] and RoBERTa [29], to (7) tokenize and fine-tune with two setups: baseline models, which were trained on only the training set, and augmented models, which were trained on both the training set and augmented data to provide a superior multi-label classification model for ambiguity detection.

4.1 Dataset Creation

In this phase, we collected software requirements from real-world projects in the PURE dataset [33], performed preprocessing to clean the data, and then created labels for four ambiguity types, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness. It is important to highlight that these types of ambiguity labels were not present in the dataset; we independently generated them using previous methods. This approach ensures no pre-existing labels are used, thus preventing potential information leakage to LMs.

4.1.1 Data Collection

We utilized the PURE dataset, which includes 79 real-world requirements documents from industrial and academic projects [33]. This dataset was suitable for our study as it contains requirements expressed in natural language, aligning with our research focus on detecting ambiguity in natural language requirements. As the projects were stored in different formats, such as PDF, DOC, HTML, and XML, and some documentation was presented in tabular format, extracting the requirements from each file was challenging. To address this issue, we utilized the text format version of the PURE dataset provided by Luitel et al. [4], which includes 40 projects that were extracted primarily through manual cleanup.

From this set, we selected seven documents in distinct domains, including weather, control systems, administration, astronomy, communication, visualization, and computer games. This diversity in project types and structures was essential for producing more generalizable results. Even within these text files,

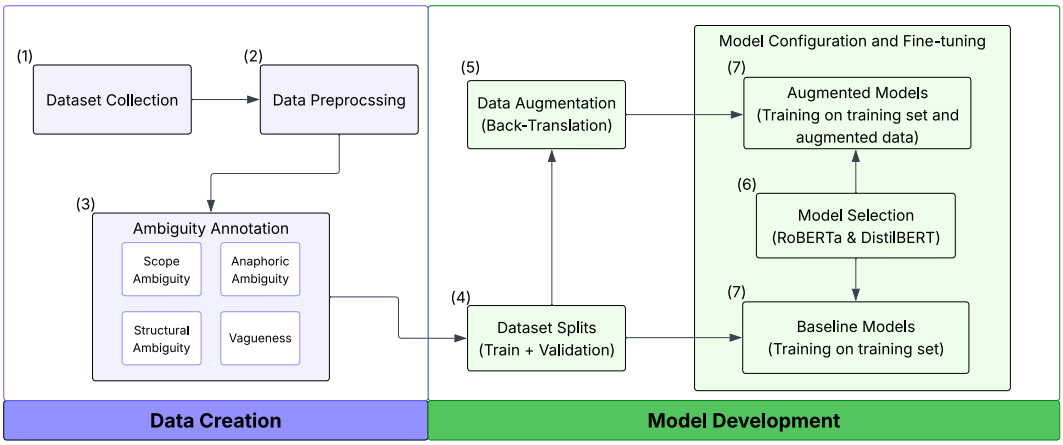


Figure 4.1: Overall Methodology for Detecting Ambiguity Types in Software Requirements

parsing the data into individual requirements required a systematic approach. Our process involved first identifying the requirements sections by finding common headers, such as Engineering Requirements, Specific Requirements, Functional Requirements, or other similar titles. Following this, we extracted individual requirements by using their unique IDs as delimiters. We developed a parsing script with a set of regular expression rules to handle the varied ID formats and separate requirements. These formats ranged from alphanumeric prefixes (e.g., F-214, UC-12) to numerical IDs (e.g., 4.1.6). For instance, in the text format of the MultiMahjong project [4], multiple requirements appeared in a single block like: "Level 1 Requirements: 4.1.1 The MultiMahjongServer will be designed to reside on a central computer (server) that is connected to a TCP/IP network and has a unique IP address. 4.1.2 The MultiMahjongServer will allow connections from MultiMahjongClients and communicate with them using IP. 4.1.3 The MultiMahjongServer will serve as a node for MultiMahjongClients. That is, it will be able to relay data received from one MultiMahjongClient to other MultiMahjongClients that request it". Our script was designed to identify the numerical IDs (e.g., 4.1.1, 4.1.2) within such blocks and correctly separate the text into three distinct requirements. This capability was crucial for accurately isolating each requirement before the manual verification stage.

This semi-automated extraction was followed by essential manual verification, as we found that some requirements documented by Luitel et al. [4] did not exactly match those in the original PURE dataset. Several sentences had been incorrectly merged, resulting in incorrect requirements in our dataset. For instance, in the Clarus Low project, the original requirement "F-214D1 CS shall accept surface condition data derived from surface image" was inaccurately documented as "F-214D1 CS shall accept surface condition "covered" or "flooded"". Table 4.1 provides additional examples of inconsistencies between the original

requirements from the PURE dataset and the requirements extracted by Luitel et al. [4]. Incorrect parts are highlighted in red for better visualization in the "PURE Dataset Requirements Extracted by Luitel et al. [4]" column. These discrepancies underscore the importance of manual verification and correction to maintain data quality and achieve alignment with the original PURE dataset. Our dataset includes 1,266 natural language requirements in multiple domains.

Table 4.1: Comparison between PURE Dataset Requirements and the preprocessed version Extracted by Luitel et al.[4]

Requirements from PURE Dataset	**PURE Dataset Requirements Extracted by Luitel et al. [4]**
F-161 The Clarus system shall be able to implement quality checking rules for specific environmental situations.	F-161 The Clarus system shall be able to implement quality ChS. The rules for setting quality flags H CAS on environmental data elements checking rules for specific environmental situations.
F-161B1 The CAS shall be able to configure quality checking rules for specific environmental situations.	F- The CAS shall be able to 161B1 configure quality checking rules for specific environmental situations.
F-111C1 The CAUI shall enable a quality manager to apply manual flags to a set of observations from an individual ESS.	F-111C1 The CAUI shall enable a quality manager to apply manual flags to environmental data includes all NTCIP H 1204 data (summarized in Table 1).

4.1.2 Data Preprocessing

The preprocessing steps included removing requirement IDs, HTML, email, and other tags from the text. Numbers were replaced with the token <NUM> using regular expressions. It should be noted that stop words were retained, as some of them can potentially contribute to various types of ambiguity that the model aims to detect. Following the data collection and preprocessing, we proceeded to label the requirements based on their ambiguity types.

4.1.3 Ambiguity Annotation

In this section, we explain our approach to creating a multi-label dataset. We used an existing open-source tool, rule-based techniques, and newly combined methods to generate labels that capture different types of ambiguity, including anaphoric ambiguity, scope ambiguity, structural ambiguity, and vagueness. For each ambiguity type, we selected a single detection method based on several criteria, such as coverage of ambiguity subtypes, demonstrated performance in prior evaluations, suitability for automated processing of individual requirements, representativeness of approaches commonly used in requirements engineering, and reproducibility and public availability. The following subsections detail our process for detecting each ambiguity type:

- **Anaphoric Ambiguity:** We used the TAPHSIR tool [22], which implements an ensemble ML classifier to identify ambiguous pronouns. For each pronoun, TAPHSIR first generates analytical triples consisting of context, pronoun, and candidate antecedent. The ensemble model then analyzes each triple using two independent classifiers, including one classifier (ML_{LF}) trained on 45 traditional linguistic features, and the other (ML_{FE}) trained on contextual feature embeddings. Each of these classifiers independently predicts whether a candidate antecedent is correct, incorrect, or inconclusive for the given pronoun. Finally, TAPHSIR combines the predictions from both classifiers and applies predefined rules to their output probabilities to determine whether the pronoun exhibits anaphoric ambiguity. The model for anaphoric defection achieved a recall of 100% and an F2-score of 88.9%, prioritizing that no potential ambiguities are missed.

- **Scope Ambiguity:** Zait and Zarour [37] detect scope ambiguity when a sentence contains a quantifier from their defined blacklist, such as any, few, little, many, much, several, and some. However, detecting ambiguity based solely on the presence of a quantifier is insufficient and may produce many false positives. For example, the requirement "The system shall log some user activities" does not have scope ambiguity, as it contains only one quantifier without interacting elements. Furthermore, their blacklist omits universal quantifiers, such as all, which can also trigger scope ambiguity [3]. To establish a more precise labeling process, we adopted a rule-based heuristic identified by Kamath et al. [27]. According to their heuristic, a requirement is flagged as scope-ambiguous if it matches at least one of these patterns:

  - Quantifier–Quantifier (QQ): Contains two or more quantifiers.
  - Quantifier–Negation (QN): Contains at least one quantifier and one negation.
  - Quantifier–Adverb (QA): Contains at least one quantifier and one frequency adverb.

  This method, shown in Algorithm 1, performs word-level tokenization using spaCy [67] and lowercases each sentence, detects keywords, and returns "True" when any condition is met. Our implementation was validated using the dataset provided by Kamath et al. [27], confirming accurate identification of their intended scope-ambiguous sentences. This successful validation confirms the reliability of applying the method to label our software requirements.

- **Vagueness:** We employed a combined list of 466 vague terms derived from the QuARS tool [19] and an additional vague list provided by SREE [28]. Both sources offer lists of terms commonly associated with vagueness. Previous research (e.g., [68]) suggested expanding these lists with a quantifier like "some" for broader vagueness detection. However, since quantifiers often relate to scope ambiguity rather than vagueness, and including them might overlap with our existing scope ambiguity detection method, we refrained from adopting this extension. Other studies, such as [37, 69], identify adjectives and adverbs as the most potentially vague. Although this

Algorithm 1: Rule-based Scope Ambiguity Detection

Input: A requirement text, *requirement_text*.
Output: A boolean indicating if scope ambiguity is present, *ScopeAmbiguous* $\in$ {True, False}.

Keyword Sets:;
$QUANT \leftarrow$ {all, every, each, some, any, several, ...};
$NEG \leftarrow$ {no, none, neither, not, never};
$FREQ \leftarrow$ {always, usually, often, frequently, sometimes, ...};
procedure *DETECTSCOPEAMBIGUITY(requirement)*
  for *sentence in requirement* **do**
    tokens $\leftarrow$ Tokenize and lowercase(*sentence*);
    /* Pattern 1: Two or more quantifiers in the same sentence */
    if $count(tokens \cap QUANT) \geq 2$ **then**
      return *True*;
    end if
    /* Pattern 2: Quantifier + negation */
    if $QUANT \cap tokens \neq \emptyset$ ***and*** $NEG \cap tokens \neq \emptyset$ **then**
      return *True*;
    end if
    /* Pattern 3: Quantifier + frequency adverb */
    if $QUANT \cap tokens \neq \emptyset$ ***and*** $FREQ \cap tokens \neq \emptyset$ **then**
      return *True*;
    end if
  end for
  /* No ambiguity patterns found in any sentence */
  return *False*;
end procedure

method definitely led to 100% recall, it resulted in numerous false positives since not all adjectives and adverbs inherently cause vagueness. For example, in the requirement "The emergency button shall be red" the adjective "red" clearly defines the color without vagueness. Despite their broad coverage of such predefined term lists, they might not capture all domain-specific vagueness, highlighting an opportunity for future improvement. However, even an advanced domain-aware method integrating POS tagging and specialized domain dictionaries, such as the method proposed by Zakeri-Nasrabadi and Parsa [70], achieved only modest performance (F1-score of 40%) with lower recall for vague adjectives and adverbs. Given these limitations, we ultimately relied on the combined list of vague terms provided by QuARS and SREE to label the requirements with a vagueness type.

- **Structural Ambiguity:** For this type of ambiguity, we used SBADR [26] that detects structural ambiguity by generating multiple parse trees via Stanford CoreNLP [55] for each sentence. The method then employs four sequential filtering pipelines to systematically eliminate grammatically incorrect and redundant parse trees. These filters are based on both structure and semantic meaning and select only the highest-scoring and statistically likely interpretations. If more than one valid interpretation remains after filtering, the sentence is marked as structurally ambiguous. This method allows us to identify potential structural ambiguity in different domains. As the original implementation is not publicly available, we developed our own implementation based on the exact description in the source paper. In order to verify that our implementation was faithful, we validated it using the examples provided in the SBADR paper and confirmed that it replicated the expected outcomes.

By applying these methods, we created a dataset with five columns: *requirement text*, *structural ambiguity*, *scope ambiguity*, *anaphoric ambiguity*, and *vagueness*. Out of the 1,266 requirements analyzed, 939 were identified with structural ambiguity, 126 with anaphoric ambiguity, 221 with scope ambiguity, and 551 with vagueness. It should be noted that individual requirements may exhibit multiple ambiguity types.

4.2 Model Development

In the Model Development phase, we first split the labeled dataset into training and validation sets. To handle class imbalance, we applied data augmentation using a BT technique, targeting the minority class within each of the structural ambiguity, scope ambiguity, and anaphoric ambiguity. We selected two LMs, DistilBERT and RoBERTa, tokenized the data, and fine-tuned these two transformer-based LMs in two separate experimental settings: training on the training dataset only, and training on the training dataset plus augmented minority class data. The detailed evaluation and performance comparison of these models on an independently labeled test set are detailed in the subsequent evaluation section (Section 5).

4.2.1 Dataset Splits

The dataset was split into a training set and a validation set. This split allocated 80% (1,012 samples) for the training set and 20% (254 samples) for the validation set. We employed the iterative stratification [71], which is suitable for multi-label classification tasks. Unlike a traditional single-label stratified split, iterative stratification preserves the distribution of label combinations in different ambiguity types across both training and validation sets, assuring that rare label combinations are proportionally represented in each subset. The training set served as the primary data source for fine-tuning the LMs, allowing them to learn patterns and relationships within the ambiguity types. Meanwhile, the validation set was used for continuous monitoring of model performance on unseen data and providing early stopping criteria to prevent overfitting, thereby ensuring the model generalizes well to new requirements.

4.2.2 Data Augmentation

Since the training set was imbalanced, we employed BT to mitigate this issue. BT preserves semantic content and underlying structural patterns while making controlled lexical variations, making it highly suitable for the ambiguity classification task.

To determine the optimal intermediate language for BT, we conducted a validation experiment. While Subedi et al. [25] demonstrated that Spanish achieved the highest BLEU score in increasing requirements for the ambiguity detection task, we validated the finding against our specific dataset by comparing Spanish with three other high-performing languages, such as French, Hindi, and Dutch. We implemented the BT process using Facebook's multilingual translation model "nllb-200-distilled-600M" [72] from the NLLB-200 family, which provides high-quality translations across 200 languages. Using the Transformers library [73] pipeline with the "translation" task identifier, we executed a two-step process, first translating English requirements to the intermediate language, then back-translating to English.

To assess the quality of the augmented data, we measure BLEU [49] and METEOR [50] scores using the Python "evaluate" library [74]. The BLEU score measures translation quality based on the precision of n-gram overlaps between the back-translated text and the original, providing insight into lexical fidelity. Specifically, we computed the 4-gram BLEU score since it considers overlapping sequences of one, two, three, and four words. The METEOR score, on the other hand, combines precision, recall, and synonym matching to evaluate semantic similarity, providing a more robust assessment of meaning preservation that accounts for paraphrasing and synonymous expressions. The results for the four languages, summarized in Table 4.2, demonstrate that Spanish achieved the highest BLEU score, reinforcing its selection for our work. The augmented dataset with Spanish achieved an average BLEU score of 0.4220 and an average METEOR score of 0.7724, outperforming French, Hindi, and Dutch. These three languages had BLEU scores below 0.40 and METEOR scores below 0.77. The BLEU score above the 40%-50% threshold indicates high translation quality, suggesting that labels for the augmented requirements remain reliable [25, 75]. The METEOR score of 0.7724 demonstrates strong semantic preservation and word alignment quality

[50]. Based on these superior scores, Spanish was selected as the language for BT augmentation in our study.

While back-translation is effective as data augmentation in software requirements, several types of translation errors could potentially affect data quality, such as semantic shifts where technical terminology or modal verbs (shall, must, should) change meaning, syntactic restructuring, and domain-specific terminology inconsistencies where technical terms may be inappropriately localized. Mitigation strategies were implemented to address these concerns. First, we selected Spanish based on its superior BLEU (0.4220) and METEOR (0.7724) scores, indicating strong semantic and structural preservation. Second, we used the high-quality NLLB-200 translation model specifically designed for multilingual translation. Finally, we applied quality thresholds above the 40% BLEU benchmark for reliable translation [25, 75]. These measures help ensure that while minor translation artifacts may exist, the semantic and structural properties essential for ambiguity classification are maintained.

Augmented samples include requirements that lacked structural ambiguity and requirements that contained either anaphoric ambiguity or scope ambiguity, with an emphasis on boosting the minority labels that were underrepresented in the training dataset. This targeted technique resulted in the following advances with Spanish: (i) For structural ambiguity, non-ambiguous requirements increased from 261 to 516, while structurally ambiguous requirements grew moderately from 751 to 966. (ii) Requirements with anaphoric ambiguity and (iii) scope ambiguity doubled from 101 to 201, and from 173 to 345, respectively. (iv) Vagueness showed balanced growth, with ambiguous requirements increasing from 459 to 677. These improvements demonstrate that the minority labels for structural ambiguity, anaphoric ambiguity, and scope ambiguity approximately doubled in size. After augmentation, the dataset contained 1,482 samples, representing a 46.4% increase from the original 1,012 training samples. However, we could not achieve full balance in the dataset, since the original gaps of some types were really huge, and the requirements had multiple labels. When augmenting requirements with anaphoric ambiguity or scope ambiguity, these samples often also contained vagueness or structural ambiguity. Similarly, requirements lacking structural ambiguity frequently also lacked anaphoric ambiguity or scope ambiguity. Achieving complete balance would require more diverse augmentation techniques and extensive manual annotation.

Table 4.2: Comparison of Average BLEU Scores for Back-Translation

Language	**Average BLEU Score**	**Average METEOR Score**
Spanish	**0.4220**	**0.7724**
Dutch	0.3838	0.7525
Hindi	0.3804	0.6463
French	0.3780	0.6796

4.2.3 Model Configuration and Fine-tuning

This section details the complete configuration and training pipeline from model selections (RoBERTa and DistilBERT) and data tokenization to the specific hyperparameters used for fine-tuning.

Model Selection

The selection of appropriate LMs is crucial for the success of multi-label ambiguity detection in software requirements. Our model selection process was guided by different key criteria that address performance, accessibility, and model size.

- **Accessibility and Cost Considerations:** We prioritized freely available, open-source models that can be fine-tuned without proprietary restrictions. This criterion led us to exclude commercial models such as GPT-4 [76], despite their demonstrated capabilities in natural language understanding tasks. The use of open-source models provides reproducibility of our research and accessibility for future studies in this domain.
- **Model Size:** Model size represents another critical factor in our selection process, as it directly influences training time, memory requirements, and inference speed. Fine-tuning large language models presents significant memory challenges and requires high-memory GPUs, as modern large language models typically contain billions of parameters. For example, memory constraints prevented fine-tuning LLaMA [77] on the available GPU.

Based on these criteria, we selected two open-source, transformer-based models to create a strategic comparison between computational efficiency and robust performance.

Our first model, DistilBERT, was selected for its proven efficiency and effectiveness. It is a compact version of BERT, retaining approximately 97% of its performance while being 40% smaller and 60% faster. Previous research has already shown that DistilBERT outperforms traditional ML models, such as LR, SVM, and KNN, for ambiguity detection [59], making it a strong and practical model. The second model, RoBERTa, was chosen to test the hypothesis that more extensive pre-training leads to better performance on this challenging task. RoBERTa is pre-trained on a significantly larger dataset (160GB) using an optimized training approach.

The comparative approach allows us to directly evaluate the performance benefits of extensive pre-training (RoBERTa) against the practical advantages of a lighter, distilled model (DistilBERT) for our multi-label ambiguity detection task. Both models have also demonstrated superior performance across various text classification tasks and different domains compared to other transformer models [78, 79, 80]. They leverage the transformer encoder architecture [1], which is effective for ambiguity detection as it can capture deep contextual and syntactic patterns within text. Additionally, both models ensure reproducibility, as they are widely available and can be easily implemented by other researchers.

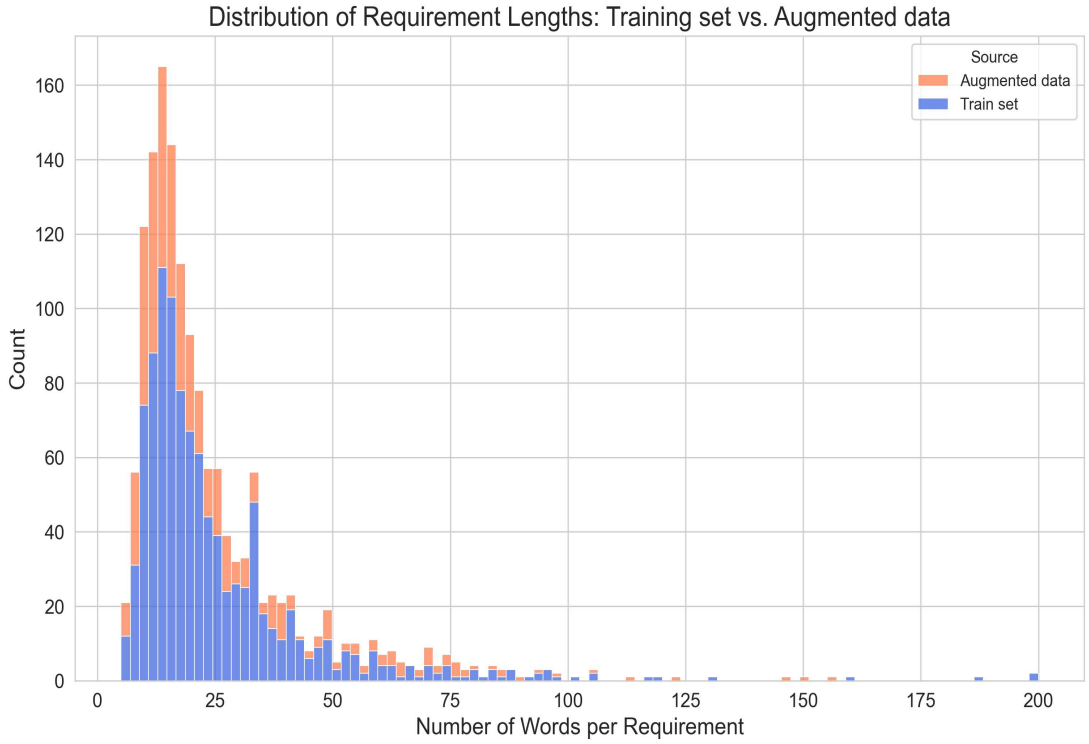


Figure 4.2: Distribution of Requirement Lengths based on Word Count (Training set and Augmented data)

Data Tokenization and Sequence Length

To prepare the data for these two models, we used two tokenizers, DistilBertTokenizer and RobertaTokenizer, both sourced from the HuggingFace Transformers library [73]. This tokenization process converts the text into numerical representations suitable for model input.

A critical hyperparameter for this process is the maximum sequence length (*max_length*). To establish a reasonable upper bound, we first performed a preliminary analysis by measuring the length of the requirements in both the training set and augmented data. Figure 4.2 shows the distribution of requirement lengths. In the histogram, the blue bars represent the frequency of the training set, while the orange bars represent the frequency of augmented data. The vast majority of requirements are well under 100 words in length, with the longest being approximately 200 words.

While the length is not identical to the token count, it provides a reliable estimate of the text's scale. Based on this analysis, and to ensure that no information would be lost for any sample after tokenization, we selected a *max_length* of 256. This value comfortably accommodates the longest texts in our dataset. To validate the choice, we analyzed the precise token length distributions using both RobertaTokenizer and DistilBertTokenizer. As shown in Figures A.1 and A.2 in Appendix A, the maximum token lengths are 248 and 251 for RoBERTa and DistilBERT tokenizers, respectively, confirming that our chosen maximum length of 256 is appropriate. For both tokenization processes, the tokenizer was configured with two key parameters:

- truncation=True: Although no data is expected to be truncated, this set-

ting is kept as a safeguard.

- padding=True: For any text shorter than 256 tokens, special [PAD] tokens are added to the end of the sequence to create uniform tensors.

The tokenization process was implemented through a function that handles both tokenizers consistently. Listing 4.1 shows the implementation code for the tokenization process. It takes the tokenizer instance and a collection of texts, applies the tokenization with the specified parameters, and returns the encoded representations as PyTorch tensors ready for model input. The conversion to tensors is essential for compatibility with the deep learning framework used in subsequent training phases.

```
1 def tokenize_text(tokenizer, texts):
2     """Tokenize text for multi-label classification"""
3     encodings = tokenizer(texts.tolist(), truncation=True,
          padding=True, max_length=256)
4     return {key: torch.tensor(val) for key, val in
          encodings.items()}
```

Listing 4.1: Text Tokenization Function for Multi-Label Classification

Model Fine-tuning and Hyperparameters

The models were fine-tuned with a learning rate of 2×10^{-5}, and a batch size of 64, chosen to provide an optimal balance between gradient stability and computational efficiency within our available GPU memory constraints. Both hyperparameters were selected based on an existing study focused on ambiguity detection using the PURE dataset [59].

Rather than adopting a fixed training duration (epoch count) from previous research [44, 59], we set $num_train_epochs = 10$ as an upper bound, and the final model was chosen based on the checkpoint that yielded the highest macro-averaged F1-score on the validation set, using the *load_best_model_at_end* parameter. This strategy assures optimal performance by automatically identifying the best stopping point during training, preventing both premature termination and overfitting that could occur in later epochs. Choosing a lower *num_train_epochs* like 2 or 3 prevents the model from fine-tuning on the labeled requirements and leads to underfitting. Empirically, the validation macro-averaged F1-score generally peaked in the early-to-mid epochs, with RoBERTa tending to peak slightly later than DistilBERT, consistent with its higher capacity. The macro-averaged F1-score was chosen as the selection metric because it provides balanced evaluation across all types, which is essential for the multi-label classification task where class imbalance may be present.

Model evaluation was performed at the end of each epoch to continuously monitor performance and enable the identification of the optimal checkpoint throughout the training process. The checkpoint management strategy retained only the 3 most recent model states to balance storage efficiency with the need for model recovery options, assuring that computational resources were used effectively while maintaining access to high-performing model variants for analysis and comparison. The hyperparameter configuration used in our code is detailed in Table 4.3.

Table 4.3: Key Hyperparameters Used for Fine-tuning

Parameter	**Value**
num_train_epochs	10
per_device_train_batch_size	64
per_device_eval_batch_size	64
learning_rate	2×10^{-5}
evaluation_strategy	epoch
save_strategy	epoch
save_total_limit	3
metric_for_best_model	f1_macro
load_best_model_at_end	True

After configuring the tokenization and model parameters, we tokenized the input texts with RobertaTokenizer and DistilBertTokenizer, and then fine-tuned the corresponding models, RobertaForSequenceClassification [81] and DistilBertForSequenceClassification [82], via the Trainer class [83] from the Transformers library [73]. Two experimental setups were employed:

- **Baseline Models:** Training was performed on the training set.
- **Augmented Models:** Training was performed on an expanded dataset, combining the training set with additional augmented samples generated via BT to address class imbalance.

For both setups, the model output was formatted as a binary prediction for each ambiguity type, indicating the presence or absence of that specific ambiguity. A sample of the model's output is the same as Table 4.4. The first column shows the *requirements text*, and the other four columns, *structural ambiguity*, *scope ambiguity*, *anaphoric ambiguity*, and *vagueness*, are the predicted output based on the model.

4.3 Summary

This section outlines a two-phase approach to developing multi-label classification models for detecting ambiguity in software requirements.

The first phase, dataset creation, involved curating a dataset of 1,266 requirements from seven distinct domains within the PURE dataset. After manual verification and preprocessing, the requirements were automatically labeled for four ambiguity types using previous methods. These methods included the TAPHSIR tool for anaphoric ambiguity, the SBADR method for structural ambiguity, a rule-based heuristic for scope ambiguity, and a combined list of vague words from the QuARS and SREE tools for vagueness.

In the model development phase, the labeled dataset was split into a training set (80%) and a validation set (20%) using iterative stratification to preserve label distributions. To mitigate class imbalance, BT was used to augment the training

Table 4.4: Examples of Multi-Label Ambiguity Classification Output

requirement text	**structural ambiguity**	**scope ambiguity**	**anaphoric ambiguity**	**vagueness**
All telescope, instrument, and detector control information is to be available at any operation level.	1	0	0	1
Transitions which occur while the train is stationary shall be initiated automatically or manually as appropriate.	0	0	0	1
Each monitor and control element has a process variable.	1	1	0	0
If the supervision is performed by a RBC it shall be possible to prevent movements of a traction unit in its area if not authorised by the RBC.	1	0	1	1
Each railway shall have the possibility of allowing a different release speed for every signal.	1	1	0	1
Partial Supervision shall be indicated on the DMI.	0	0	0	0

data, which increased the dataset by approximately 46% to 1,482 samples. The quality of the Spanish-augmented data was assessed with a BLEU score of 0.4220 and a METEOR score of 0.7724, outperforming French, Hindi, and Dutch on both metrics. Two transformer-based LMs, DistilBERT and RoBERTa, were selected for the multi-label classification task. The tokenization and model parameters included a *max_length* of 256, a batch size of 64, a learning rate of 2×10^{-5}, and early stopping enabled through the *load_best_model_at_end* parameter, based on the macro-averaged F1-score. Tokenization and model fine-tuning were performed in two experimental scenarios: baseline models trained solely on the training dataset, and augmented models trained on both the training dataset and the augmented dataset. The subsequent chapter presents the evaluation of these fine-tuned models using an independently annotated manual test set and multiple evaluation metrics.

Chapter 5

Evaluation

We validate our approach by creating a manually annotated test set, defining different evaluation metrics, and presenting experimental results that address three research questions. By comparing our models to the test set, we can obtain a more reliable and objective assessment of their actual performance.

5.1 Manual Test Set

The annotated test set contained 255 requirements from the PURE dataset, which were not included in the training set and validation set. To ensure the reliability and quality of labels, the labeling was performed manually by three annotators. Two annotators were experts in requirements engineering (including the author of the thesis), while the third annotator was a linguistics expert specializing in sentence structure and interpretation. Each annotator assigned binary labels (where '1' indicates the presence of ambiguity and '0' reveals its absence) for each of the four ambiguity types, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness.

Table 5.1: Fleiss' Kappa Value for Each Type of Ambiguity

Ambiguity Types	**Fleiss' Kappa Value**
Structural Ambiguity	0.6131
Scope Ambiguity	0.6147
Anaphoric Ambiguity	0.7255
Vagueness	0.5077
Overall	**0.6153**

In order to obtain our final labels, we employed majority voting. For each ambiguity type per requirement, if at least two annotators assigned the same label, it was adopted as the definitive label. For example, when labeling a requirement for structural ambiguity, if two annotators assigned label 1 and one assigned label 0, we adopted label 1 as the final label. To measure the consistency among annotators, we calculated Fleiss' kappa value [52] for each ambiguity type and then reported the average agreement across all types. Following the accepted interpretation scale proposed by Landis and Koch [53], a kappa value between 0.41 and 0.60 indicates moderate agreement, while a value from 0.61 to 0.80 indicates substantial agreement. Based on Table 5.1, structural ambiguity, anaphoric ambiguity, and scope ambiguity achieved substantial agreement; however, vagueness showed moderate agreement due to its subjective nature and reliance on

contextual interpretation. The overall average kappa score across all types was 0.6153, indicating a substantial agreement range. This result confirms that our annotation process produced reliable labels suitable for model evaluation.

Finally, to maintain compatibility between the training set and testing set, we applied the same preprocessing steps described in Section 4.1.2 to the 255 annotated requirements. This process involved removing requirement IDs, eliminating unnecessary tags, and masking numerical values. Therefore, these steps maintained consistency between the test data format and the training data format used by the models.

5.2 Evaluation Metrics

To evaluate our multi-label classification model for ambiguity type detection, we use two distinct categories of metrics: (i) individual ambiguity type performance and (ii) multi-label performance.

5.2.1 Individual Ambiguity Type Performance

We compare our approach with existing ambiguity detection methods using accuracy, precision, recall, and F1-score, which are the standard metrics in similar studies [24, 59]. We also provide confusion matrices for each type to present detailed insights into classification performance. This comparison provides a precise assessment of our unified model relative to existing specialized methods.

For each ambiguity type i, the metrics are computed as follows, where TP_i, FP_i, FN_i, and TN_i represent true positives, false positives, false negatives, and true negatives, respectively. The standard metrics range from 0 to 1, and the higher values indicate better performance.

Accuracy: The proportion of correct predictions, both positive and negative, among all predictions for ambiguity type i:

$$\mathrm{Accuracy}_i = \frac{\mathrm{TP}_i + \mathrm{TN}_i}{\mathrm{TP}_i + \mathrm{TN}_i + \mathrm{FP}_i + \mathrm{FN}_i}. \tag{5.1}$$

Precision: The proportion of correct positive predictions among all positive predictions for ambiguity type i:

$$\mathrm{Precision}_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FP}_i}. \tag{5.2}$$

Recall: The proportion of true positives identified correctly among all actual positive cases for ambiguity type i:

$$\mathrm{Recall}_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FN}_i}. \tag{5.3}$$

F1-score: The harmonic mean of precision and recall, providing a balanced performance measure for ambiguity type i:

$$F1\text{-score}_i = 2\frac{\mathrm{Precision}_i \cdot \mathrm{Recall}_i}{\mathrm{Precision}_i + \mathrm{Recall}_i}. \tag{5.4}$$

Confusion Matrix: For each ambiguity type i, the confusion matrix is presented as a 2×2 table that summarizes the classification performance by comparing predicted labels against actual labels. The matrix displays the counts of TP_i, FP_i, TN_i, and FN_i, providing a clear view of how well the model distinguishes between ambiguous and non-ambiguous requirements for each specific ambiguity type. This representation helps us to identify specific patterns of misclassification and understand the model's behavior across different ambiguity categories.

By examining these performance metrics, including accuracy, precision, recall, F1-score, and confusion matrix, for each ambiguity type separately, this evaluation approach leads to quantitatively identifying our model's strengths and weaknesses for structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness independently. This approach also reveals whether certain ambiguity types are more challenging to detect than others. The individual assessment is essential for addressing RQ3, as it clarifies the trade-offs between employing a unified multi-label model against multiple specialized detection methods.

5.2.2 Multi-Label Performance

We compute macro-averaged precision, recall, and F1-score across all ambiguity types to evaluate the overall performance of multi-label classification models [84, 85]. For example, the recall of the multi-label classification model is calculated as the macro average of individual recall scores from each ambiguity type. Macro averaging weights every type equally, ensuring that rare ambiguity types contribute equally to the overall evaluation as more frequent ones [86, 87]. Macro-averaged precision, recall, and F1-score are calculated as in equation (5.5):

$$\text{Macro-Averaged Metric} = \frac{1}{L}\sum_{i=1}^{L} M_i, \quad M \in \{\text{Precision},\ \text{Recall},\ \text{F1-score}\}, \tag{5.5}$$

where L is the number of ambiguity types and M_i is the metric value for type i.

Accuracy is not reported for the performance of the multi-label classification models because it requires all labels per instance to be correct, which fails to distinguish between false positive and false negative errors and can mask performance issues with rare labels [88]. Instead, hamming accuracy is used because it evaluates each label independently and then averages the results, providing equal weight to all ambiguity types regardless of their frequency in the dataset. This is particularly important for requirements ambiguity detection, where rare ambiguity types may be just as critical to identify as common ones. Hamming accuracy measures the proportion of correctly predicted labels across all samples and is widely adopted for multi-label evaluation [89]. Hamming accuracy is calculated as in equation (5.6):

$$\text{Hamming Accuracy} = 1 - \frac{1}{NL}\sum_{n=1}^{N}\sum_{i=1}^{L}\big[\,\hat{y}_{n,i} \neq y_{n,i}\big], \tag{5.6}$$

where N is the number of requirements, L is the number of ambiguity types, $y_{n,i}$ the actual label, and $\hat{y}_{n,i}$ the prediction for type i on instance n.

Furthermore, the confusion matrix, suitable for single-label or multi-class scenarios, cannot be applied to multi-label classification [90]. In single-label classification, each instance belongs to exactly one class, making it straightforward to distinguish "true class" from "predicted class". In contrast, multi-label classification allows each instance to have multiple true and predicted labels at the same time, complicating the direct application of the confusion matrix.

This dual evaluation framework, combining (i) individual ambiguity type performance and (ii) multi-label performance, is essential for ambiguity detection research. The individual metrics facilitate direct benchmarking against existing methods, while the multi-label metrics assess the model's performance in realistic scenarios where requirements often have multiple forms of ambiguity.

5.3 Results and Analysis

In this section, we outline the methodology used to address each research question and report the associated results.

5.3.1 Which fine-tuned language model achieves the highest baseline performance for multi-label ambiguity detection? (RQ1)

We compare the baseline performance of the DistilBERT and RoBERTa models after each was fine-tuned on the training set and evaluated on a manually annotated test set.

Table 5.2 shows the macro-averaged metrics and hamming accuracy of the two models. RoBERTa attains a higher macro-averaged F1-score of 75.61%, outperforming DistilBERT by about 2.5%. In addition, RoBERTa has better performance across all evaluated metrics, including a higher macro-averaged precision of 67.68%, macro-averaged recall of 88.24%, and hamming accuracy of 83.17%, indicating a consistent improvement over DistilBERT.

Higher macro-averaged F1-score shows that RoBERTa effectively balances precision and recall, which is important for identifying ambiguous requirements without incorrectly labeling clear requirements as ambiguous. The improved precision implies fewer false positives, meaning RoBERTa was more accurate in classifying requirements correctly as ambiguous, while the higher recall indicates fewer missed ambiguous requirements.

The enhanced performance of RoBERTa can be attributed to the model's capacity and pre-training advantages. It retains about 125 million parameters and 12 encoder layers, compared to DistilBERT's 66 million parameters and 6 encoder layers. This larger capacity gives greater representational power to capture complex linguistic patterns essential for ambiguity detection. Furthermore, RoBERTa was pre-trained on more than 160GB of diverse text with key training optimizations including dynamic masking and longer sequence lengths, which helps it extract more robust features and generalize better to unseen requirements.

Table 5.2: Comparison of DistilBERT and RoBERTa in Detecting Ambiguity Types

Metrics (%)	DistilBERT	RoBERTa
Precision	65.33	**67.68**
Recall	85.14	**88.24**
F1-score	73.15	**75.61**
Hamming Accuracy	81.30	**83.17**

Note: Precision, Recall, and F1-score are macro-averaged.

RQ1 Conclusion

Fine-tuned RoBERTa appears to be a more effective baseline model for multi-label ambiguity detection than fine-tuned DistilBERT. The macro-averaged F1-score for RoBERTa is 75.61% compared to DistilBERT's 73.15%, and the hamming accuracy is improved by approximately 2%. This consistently better performance across all evaluation metrics reflects RoBERTa's ability to more accurately distinguish between ambiguous and clear requirements. The performance advantage arises from RoBERTa's greater model complexity, richer contextual understanding, and extensive pre-training, making it especially suitable for the complex task of ambiguity detection.

5.3.2 Which language model provides the best performance after being fine-tuned with BT augmentation? (RQ2)

To evaluate the impact of data augmentation, we fine-tuned DistilBERT and RoBERTa using an expanded training set, incorporating additional back-translated samples (augmented models). These new samples were generated using BT, focusing on underrepresented labels within structural ambiguity, scope ambiguity, and anaphoric ambiguity, as described in Section 4.2.2. The results demonstrate that RoBERTa remains the superior model after augmentation.

Table 5.3: Comparison of DistilBERT and RoBERTa in Detecting Ambiguity Types after Back-Translation Augmentation

Metrics (%)	DistilBERT	RoBERTa
Precision	66.27	**69.79**
Recall	85.08	**90.69**
F1-score	73.81	**78.84**
Hamming Accuracy	82.68	**85.24**

Note: Precision, Recall, and F1-score are macro-averaged.

As shown in Table 5.3, RoBERTa achieves a macro-averaged F1-score of 78.84%, marking an improvement of 5.03% over DistilBERT's score of 73.81%. Beyond the F1-score, RoBERTa consistently surpasses DistilBERT across other metrics, including macro-averaged precision, macro-averaged recall, and hamming accuracy. Specifically, RoBERTa's macro-averaged precision of 69.79% indicates a superior ability to correctly identify ambiguous requirements without excessive false positives. A macro-averaged recall of 90.69% for RoBERTa signifies that the model captures a higher proportion of actual ambiguous cases compared to DistilBERT. Furthermore, RoBERTa's higher hamming accuracy (85.24%) emphasizes its effectiveness in accurately labeling each ambiguity type within the multi-label context. These improvements clearly establish RoBERTa's enhanced overall performance relative to DistilBERT.

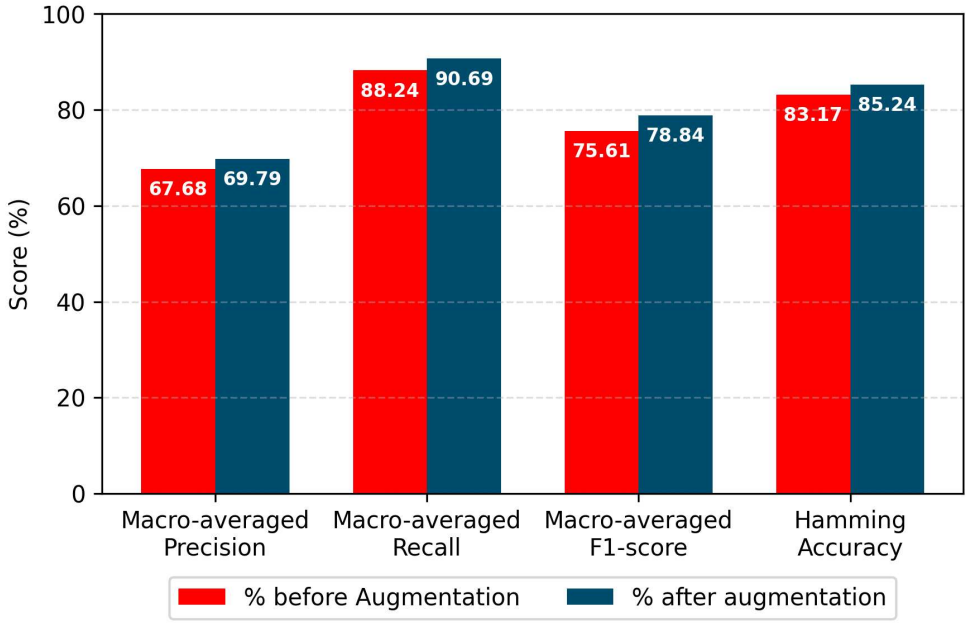


Figure 5.1: Comparison of RoBERTa Model before and after Data Augmentation with Back-translation

Figure 5.1 further illustrates the effectiveness of BT augmentation specifically for RoBERTa as the superior model, highlighting performance gains compared to its baseline results from RQ1. RoBERTa's macro-averaged precision improved from 67.68% to 69.79%, macro-averaged recall from 88.24% to 90.69%, macro-averaged F1-score from 75.61% to 78.84%, and hamming accuracy from 83.17% to 85.24%. Given our objective to improve recall and minority-label coverage, RoBERTa gains justify the additional preprocessing effort and extended training time required for BT. The improved performance better aligns predicted labels with manual test set annotations. In contrast, the augmented DistilBERT model exhibited only minimal improvement compared to DistilBERT baseline performance from RQ1, with the F1-score increasing marginally from 73.15% to 73.81% (0.66%). This negligible improvement suggests BT may not be cost-effective for the smaller model with limited computational budgets. RoBERTa's superior capacity enables it to better leverage the augmented data for detecting minority labels within structural ambiguity, anaphoric ambiguity, and scope am-

biguity compared to the lightweight DistilBERT model. Thus, the combination of RoBERTa and BT augmentation is identified as the most effective approach for the multi-label ambiguity detection task in this study.

RQ2 Conclusion

By adding augmented data to the training process of both RoBERTa and DistilBERT, RoBERTa demonstrated superior results in multi-label ambiguity detection across all evaluation metrics. These improvements also show that RoBERTa consistently outperforms the best-performing baseline model evaluated in RQ1. The observed improvements confirm that applying targeted data augmentation effectively strengthens RoBERTa's overall performance and reliability.

Table 5.4: Performance Comparison Between Proposed and Existing Methods for Each Ambiguity Type

Ambiguity Types	**Metrics (%)**	**Existing Works**	**Model**
Structural Ambiguity	Accuracy	56.30	74.02
	Precision	48.91	63.76
	Recall	84.11	88.79
	F1-score	61.86	**74.22**
Scope Ambiguity	Accuracy	90.16	93.31
	Precision	60.00	70.21
	Recall	91.67	91.67
	F1-score	72.53	**79.52**
Anaphoric Ambiguity	Accuracy	98.43	98.03
	Precision	84.62	81.48
	Recall	100	100
	F1-score	**91.67**	89.80
Vagueness	Accuracy	78.74	75.59
	Precision	67.80	63.71
	Recall	83.33	82.29
	F1-score	**74.77**	71.82

Note: The existing methods' results for each ambiguity type are from different, specialized studies.

5.3.3 How does our unified multi-label model, the best-performing model from RQ2, compare to previous detection methods? (RQ3)

We compared our unified multi-label model (the best model performance from RQ2) against four existing methods. For structural ambiguity, we compare with

SBADR [26], as it covers all structural ambiguity subtypes, such as attachment ambiguity, coordination ambiguity, and analytical ambiguity, and employs a robust detection mechanism based on multiple parse tree analysis. Other existing works may cover only a subset of structural ambiguity or lack the evaluation analysis to enable a reliable comparison [56, 57, 65]. For anaphoric ambiguity, we selected the TAPHSIR tool [22] as it represents the current state-of-the-art method for detecting anaphoric ambiguity, achieving 100% recall performance in their evaluation. For scope ambiguity, we compare with the rule-based heuristic method [27], as it employs systematic pattern matching (QQ, QN, QA patterns) rather than relying solely on predefined keyword lists that may miss potential scope ambiguity [37, 18]. For vagueness detection, we use the combined vague word lists from QuARS and SREE [19, 28], since alternative approaches either overlap significantly with other ambiguity types [68], are applicable only to other languages such as Thai [57], or demonstrate insufficient performance with recall scores as low as 48.85% [70].

Table 5.4 presents different comparisons between our model and existing methods. For each type, we evaluate performance using the F1-score, recall, precision, accuracy, and confusion matrices. Two confusion matrices are provided per ambiguity type: one representing our model and another representing the existing specialized method. Both matrices use the same green gradient with a consistent scale to facilitate direct visual comparison. Darker shades indicate higher counts within the matrices. The following sections present a detailed analysis of each ambiguity type individually.

- **Structural Ambiguity:** Our multi-label model significantly outperforms the SBADR method, achieving a higher F1-score of 74.22%, compared to SBADR's 61.86%. This notable difference of 12.36% is consistently reflected across all evaluation metrics, especially accuracy, where our model achieves 74.02% compared to SBADR's 56.30%. Moreover, our model demonstrates higher precision (63.76% vs. 48.91%) and recall (88.79% vs. 84.11%), indicating superior performance in correctly identifying structurally ambiguous requirements while reducing classification errors.

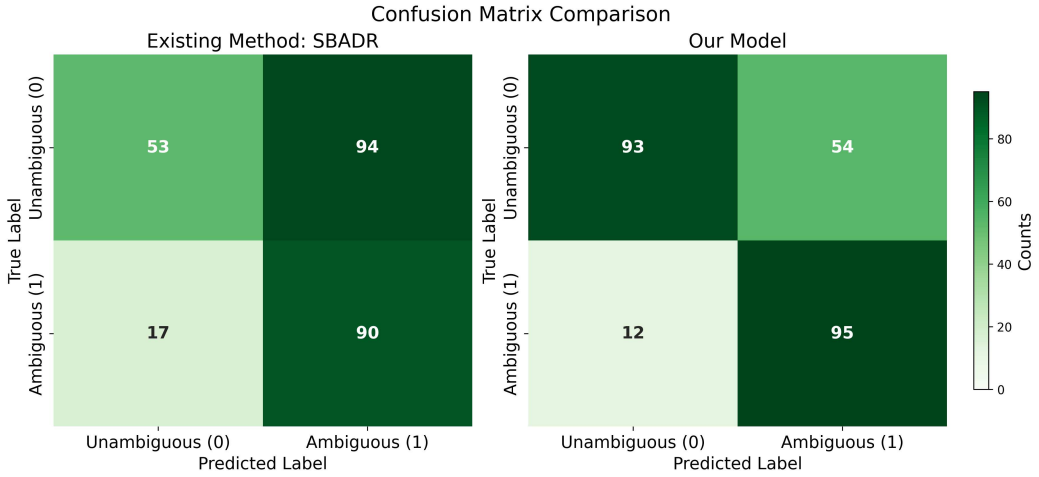


Figure 5.2: Confusion Matrices for Structural Ambiguity

The confusion matrices are shown in Figure 5.2 to further highlight these

improvements. The SBADR method incorrectly classifies a large number of clear requirements as ambiguous, producing 94 false positives, while missing several ambiguous cases, resulting in 17 false negatives. In contrast, our model achieves both fewer false positives (54) and false negatives (12), demonstrating a more balanced and accurate classification performance. This improvement of our model is notable given the increased number of requirements that do not have this type of ambiguity, reducing false positives compared to the existing method. Therefore, the proposed unified model is more effective at correctly identifying structural ambiguity compared to the specialized SBADR method.

- **Scope Ambiguity:** As shown in Table 5.4, the RoBERTa model attains an F1-score of 79.52%, surpassing the rule-based method's score of 72.53%. Our model also achieves higher accuracy (93.31% vs. 90.16%) and precision (70.21% vs. 60.00%) compared to the existing method. Both models share the same recall (91.67%), indicating our model maintains the same high level of sensitivity in detecting ambiguous cases.

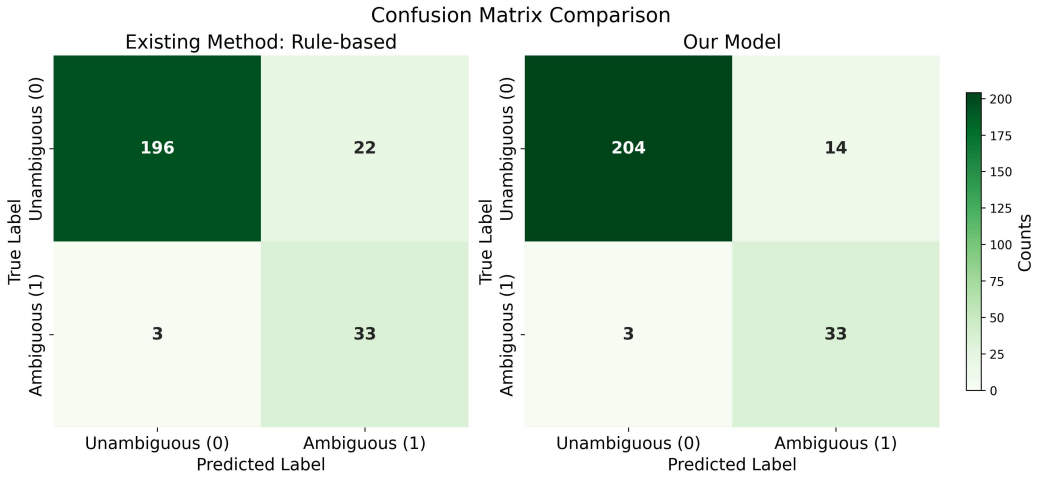


Figure 5.3: Confusion Matrices for Scope Ambiguity

Confusion matrices in Figure 5.3 further emphasize these performance enhancements. The existing rule-based method produced 22 false positives, whereas our model produced 14 false positives, clearly demonstrating its ability to better differentiate clear requirements from ambiguous ones. Both models miss only 3 ambiguous cases, reinforcing that the improvement lies primarily in reducing false alarms. Consequently, the proposed multi-label model provides a more reliable and precise solution for identifying scope ambiguity compared to the specialized rule-based method.

- **Anaphoric Ambiguity:** In detecting anaphoric ambiguity, our multi-label model achieves very close performance compared to the existing tool, TAPHSIR, as presented in Table 5.4. The TAPHSIR tool attains a slightly higher F1-score of 91.67%, compared to our model's 89.80%. Both models also achieve very close accuracy, with TAPHSIR at 98.43% and our model at 98.03%. The model and tool both demonstrate a recall of 100%, indicating they correctly detect all requirements containing this type of ambiguity.

However, TAPHSIR shows slightly higher precision (84.62%) compared to our model (81.48%), resulting in its marginally superior overall F1-score.

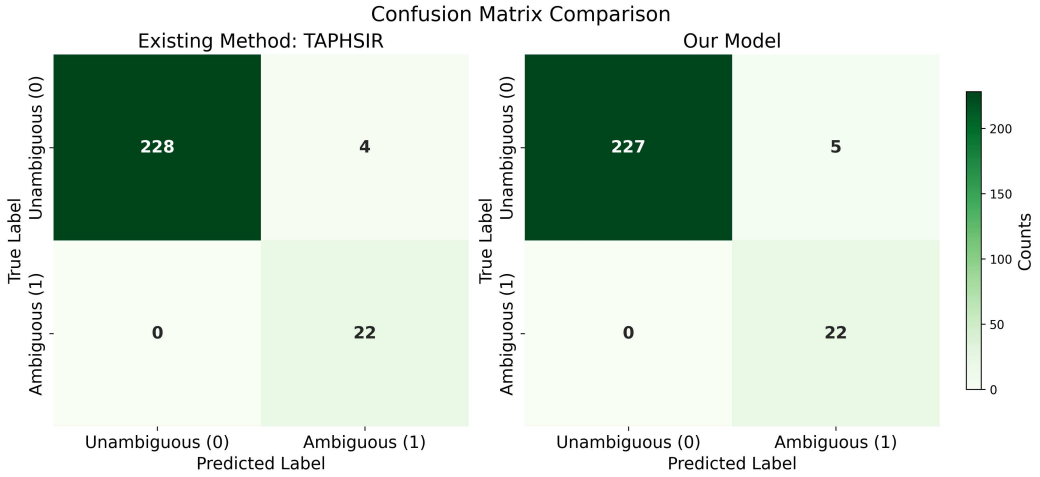


Figure 5.4: Confusion Matrices for Anaphoric Ambiguity

The confusion matrices in Figure 5.4 provide additional insights. TAPHSIR incorrectly identifies 4 clear requirements as ambiguous (false positives), whereas our model slightly increases this to 5. This minimal difference, just one additional incorrect label, results in a slightly lower precision for our model. Nonetheless, our model and the tool achieve zero false negatives, ensuring that no cases of anaphoric ambiguity were missed. Thus, while TAPHSIR marginally outperforms our unified model in precision due to this minor difference, our model still delivers strong, comparable results, maintaining a high standard within a unified multi-label detection approach.

- **Vagueness:** For detecting vagueness, our unified multi-label model achieves a slightly lower F1-score of 71.82% compared to the existing method's score of 74.77%, as detailed in Table 5.4. The model gains a precision of 63.71% against the existing method's 67.80%, while recall remains relatively similar (82.29% for our model vs. 83.33% for the existing method). Accuracy follows a similar pattern, with our model achieving 75.59% compared to the existing method's 78.74%.

  The confusion matrices presented in Figure 5.5 further highlight these differences. Our model generates more false positives (45 compared to 38) and just one additional false negative (17 vs. 16), resulting in the observed lower precision and recall. This outcome likely arises because our model depends on contextual semantic understanding rather than strictly flagging predefined terms. Although this provides the benefit of broader coverage, it can occasionally result in more false-positive errors and missed cases compared to the word-list method. Moreover, the model might need more varied training data to better understand vague language and perform better in this area, as vagueness takes so many different forms that the stacked self-attention layers cannot consistently retain their subtle cues. Despite a slight 3% decline in F1-score, our model misclassifies only one

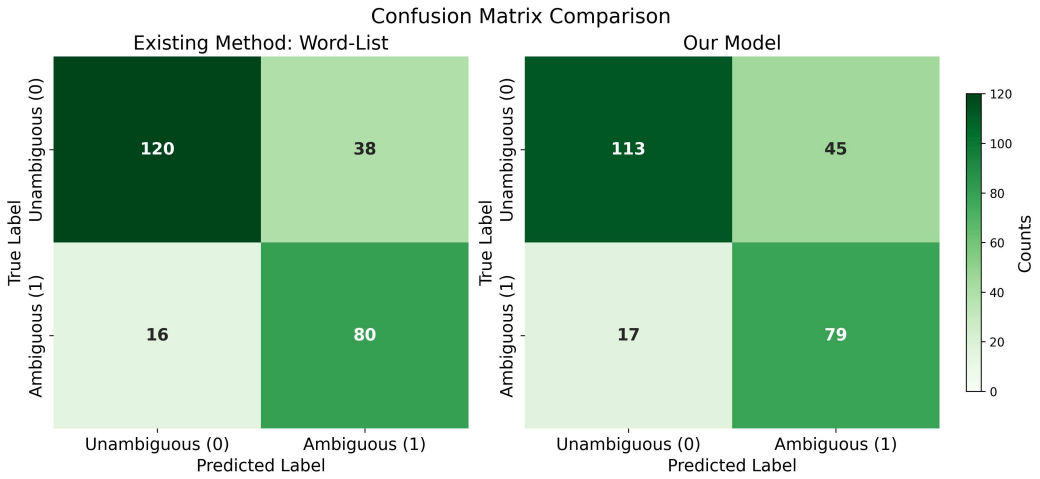


Figure 5.5: Confusion Matrices for Vagueness

additional vague requirement compared with the existing method, indicating mostly comparable coverage of vagueness while still delivering balanced performance across all four ambiguity types.

Based on our results, the unified multi-label model provides a highly effective approach for detecting structural ambiguity and scope ambiguity. For anaphoric ambiguity and vagueness, our model performs slightly lower than specialized methods by about 2% and 3% in F1-score, respectively. However, recall scores are nearly identical, with the same recall for anaphoric ambiguity and differing by only one misclassified requirement for vagueness, showing that our model identifies almost the same ambiguous cases. This trade-off underscores the practical value of our unified approach. Prior methods typically target only one or two ambiguity types, necessitating the implementation of multiple disparate methods for broader coverage. For instance, detecting both anaphoric ambiguity with TAPHSIR and structural ambiguity with SBADR requires managing methods built in different programming languages (e.g., Python and Java), along with their separate dependencies and output formats. Integrating these disparate outputs increases complexity and resource demands. Accordingly, our unified model simplifies this process by identifying multiple ambiguity types within the existing approaches, providing a more practical and efficient solution. Given the effectiveness of LMs for ambiguity detection, we anticipate further performance improvements through fine-tuning larger capacity models, such as large language models.

RQ3 Conclusion

Our unified multi-label model, leveraging BT-augmented fine-tuned RoBERTa, demonstrates superior performance compared to existing specialized methods for structural ambiguity and scope ambiguity, while achieving comparable results for anaphoric ambiguity and vagueness. Since our model achieves nearly identical performance to existing methods in detecting anaphoric ambiguity and vagueness, it provides reliable detection across all ambiguity types. The unified multi-label approach effectively simplifies the ambiguity types detection process, providing strong and balanced overall performance across multiple ambiguity types.

5.4 Summary

This chapter details the evaluation of a unified, multi-label model for detecting ambiguity types in software requirements. The evaluation is based on a manually annotated test set of 255 requirements, labeled for structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness by three experts, achieving substantial inter-annotator agreement. Subsequently, the chapter details the evaluation metrics used to assess model performance. In the baseline comparison (RQ1), fine-tuned RoBERTa consistently outperforms fine-tuned DistilBERT across all evaluation metrics. This superior performance results from RoBERTa having greater model complexity, richer contextual understanding, and extensive pre-training on diverse corpora. Incorporating BT data augmentation (RQ2) further enhanced RoBERTa's predictive capabilities and reliability, confirming it as the best-performing model. Finally, we compared the best-performing model to specialized methods (RQ3), demonstrating its effectiveness in handling multiple ambiguity types, with robust results for structural ambiguity, scope ambiguity, and comparable performance for anaphoric ambiguity and vagueness. This evaluation clearly underscores the practical and methodological advantages of employing a unified, multi-label approach. The following chapter presents the overall conclusions, discusses key challenges encountered, and outlines implications for future research.

Chapter 6

Conclusion and Future Work

This research enhances the automatic detection of different ambiguity types in software requirements to significantly reduce misunderstandings among project teams and stakeholders, thereby improving software quality [9]. Our approach identifies multiple ambiguity types, including structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness, overcoming limitations associated with previous detection methods. The dataset comprises requirements from diverse domains within the PURE dataset. These requirements were labeled using established methods and utilized to fine-tune two transformer-based LMs, DistilBERT and RoBERTa, as baseline models. The models were then further fine-tuned using additional data generated via BT augmentation to reduce class imbalance. For evaluation, we assessed the models using a manually annotated test set of 255 requirements labeled by three experts. The annotations achieved an average Fleiss' kappa score of about 0.62, indicating substantial annotator agreement.

Comparing the baseline models, fine-tuned RoBERTa, which was pre-trained on a larger corpus, outperformed fine-tuned DistilBERT. After incorporating BT augmentation, RoBERTa continued to achieve the highest performance, with a macro-averaged F1-score of 78.84%. This final model demonstrated considerable improvement, or performed comparably to existing methods, in detecting structural ambiguity, scope ambiguity, anaphoric ambiguity, and vagueness.

6.1 Threats to Validity

Despite the promising results, several factors could threaten the validity of this study.

Potential False Positives in the Dataset: Our study relies on a dataset where labels were generated using a combination of existing methods. While our final model achieved a high macro-averaged recall of 90.69%, macro-averaged precision was comparatively lower at 69.79%. This difference between high recall and lower precision is explained by the detailed results for each ambiguity type presented in Table 5.4. The model's precision is notably lower for structural ambiguity (63.76%) and vagueness (63.71%), thereby impacting the overall macro-average precision. This discrepancy is likely attributable to the limitations of the initial labeling methods, such as SBADR for structural ambiguity and predefined word lists for vagueness, which possess limited precision. Consequently, our model, during fine-tuning, may have inherited these labeling inaccuracies. While the primary objective of this study was to assess the LMs' capability to detect ambiguity types given a dataset, a more precise examination of the training dataset is necessary to maintain its high quality for future applications.

Dataset Imbalance: As discussed in 4.2.2, the dataset exhibits a class imbalance. While BT augmentation was employed to mitigate this issue, it did not

completely resolve the imbalance. Ambiguity types like structural ambiguity and vagueness are naturally more prevalent in real-world requirements. The natural uneven distribution within the dataset may still influence the model's learning process, potentially leading to a bias towards more frequent classes and impacting performance on less-represented ambiguity types. A more balanced dataset would be beneficial in addressing this limitation.

Hyperparameters: Optimal hyperparameter tuning is a complex process that often requires dedicated research to achieve optimal performance. In this study, we adopted hyperparameters from existing work within the same domain. Although efforts were made to enhance the training process by incorporating the *load_best_model_at_end* parameter based on the F1-score, which determines the effective number of epochs, the selected hyperparameters may not necessarily be optimal.

Model Scope: Both models, RoBERTa and DistilBERT, are English-centric encoder transformers; results may not transfer to other languages without multilingual pretraining and fine-tuning requirements. Since most requirements are written in natural-language text, we focus exclusively on this format and do not address alternative representations such as Unified Modeling Language (UML) diagrams and formal modelling languages.

Internal Validity: Annotator bias represents a potential threat to the internal validity of our evaluation. To mitigate this risk, three individuals (including the author of this thesis) participated in the annotation process to minimize the potential influence of any single annotator's subjective interpretation. However, achieving complete elimination of bias in human annotation is challenging, and subtle influences on the labeling process could still exist.

6.2 Future Work

There are some opportunities for further research and improvement to enhance the model's performance, expand its applicability, and facilitate its integration into real-world software engineering practices.

6.2.1 Dataset Refinement and Expansion

According to Domingos [91], the quality and quantity of data are crucial for achieving superior model performance. Therefore, enhancing labeling precision and data diversity may yield improved outcomes.

Enhanced Labeling Precision:

The observed lower precision for structural ambiguity and vagueness, resulting from limitations of initial labeling methods, necessitates a dedicated effort to refine these labels. Future work could engage domain experts involved in the training process to systematically re-evaluate and correct existing labels for these underperforming ambiguity types.

Dataset Diversity:

Expanding datasets to include more diverse domains is essential for improving model generalizability. Incorporating requirements from a broader range of industrial and academic projects beyond the current seven domains would help the model adapt to different linguistic styles. Moreover, we could explore extending the approach to requirements expressed in other formats, such as UML diagrams, use case specifications, or semi-formal notations. However, this expansion would require developing new preprocessing pipelines and potentially different ambiguity detection approaches.

Dataset Balance:

Future work will address class imbalance through different label-preserving augmentations (e.g., controlled paraphrase generation) and by having software engineers create new requirements that specifically include underrepresented ambiguity types. This is particularly important since anaphoric ambiguity and scope ambiguity are minority types that occur less frequently than the naturally more common vagueness and structural ambiguity in real-world requirements.

6.2.2 Expanding Ambiguity Type Coverage

Our study excluded certain ambiguity types due to limitations in existing detection methods or the necessity of additional contextual information beyond our current scope. Future work could aim to extend the model's capabilities to include lexical ambiguity, pragmatic ambiguity, and language errors. Pragmatic ambiguity detection can be improved by providing the model with project-specific documentation as context. Lexical ambiguity detection can be enhanced by combining WordNet with masking techniques, allowing labeling of ambiguous lexical items. Finally, identifying patterns or rules for grammatical errors could generate new labels, allowing for further fine-tuning of the model and expanding its ability to detect a broader range of ambiguities.

6.2.3 Integration into Interactive Feedback Systems

The primary goal of ambiguity detection is to move beyond passive analysis and actively aid in the creation of higher-quality requirements. Future work should focus on integrating the detection model into an interactive feedback system. Such a tool could provide real-time feedback to requirements authors, highlighting potential ambiguities in software requirements and offering context-aware suggestions for improvement. This would empower engineers to address issues proactively rather than reactively, improving the requirements of the engineering process. Achieving this vision could involve leveraging advanced prompt engineering with large language models to not only detect the specific parts that cause ambiguity but also recommend resolutions.

6.2.4 Hyperparameter Optimization

While we utilized *load_best_model_at_end* and adopted hyperparameters from existing work, a systematic hyperparameter optimization study using techniques

like Bayesian optimization could identify specific optimal configurations for the chosen LMs. However, it is important to acknowledge that even advanced methods may not fully guarantee the absolute global optimum for all hyperparameters.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] M. Q. Riaz, W. H. Butt, and S. Rehman, "Automatic detection of ambiguous software requirements: An insight," in *2019 5th International Conference on Information Management (ICIM)*. IEEE, 2019, pp. 1–6.

[3] D. M. Berry, E. Kamsties, and M. M. Krieger, "From contract drafting to software specification: Linguistic sources of ambiguity," University of Waterloo, Handbook (Technical Report), 2003. [Online]. Available: https://cs.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf

[4] D. Luitel, S. Hassani, and M. Sabetzadeh, "Improving requirements completeness: Automated assistance through large language models," *Requirements Engineering*, vol. 29, no. 1, pp. 73–95, 2024.

[5] N. Carlson and P. Laplante, "The NASA automated requirements measurement tool: a reconstruction," *Innovations in Systems and Software Engineering*, vol. 10, no. 2, pp. 77–91, 2014.

[6] A. A. Alshazly, A. M. Elfatatry, and M. S. Abougabal, "Detecting defects in software requirements specification," *Alexandria Engineering Journal*, vol. 53, no. 3, pp. 513–527, 2014.

[7] A. Rani and G. Aggarwal, "Advanced practices to detect ambiguities and inconsistencies from software requirements," in *2018 International Conference on System Modeling & Advancement in Research Trends (SMART)*. IEEE, 2018, pp. 17–21.

[8] D. M. Berry and E. Kamsties, "Ambiguity in requirements specification," in *Perspectives on software requirements*. Springer, 2004, pp. 7–44.

[9] E. Kamsties, D. M. Berry, B. Paech, E. Kamsties, D. Berry, and B. Paech, "Detecting ambiguities in requirements documents using inspections," in *Proceedings of the first workshop on inspection in software engineering (WISE'01)*, vol. 13. Citeseer, 2001.

[10] E. Euler, "The failures of the Mars Climate Orbiter and Mars Polar Lander-A perspective from the people involved," in *Guidance and control 2001*, 2001, pp. 635–655.

[11] R. Van Megen and D. B. Meyerhoff, "Costs and benefits of early defect detection: experiences from developing client server and host applications," *Software Quality Journal*, vol. 4, pp. 247–256, 1995.

[12] L. Zhao, W. Alhoshan, A. Ferrari, K. J. Letsholo, M. A. Ajagbe, E.-V. Chioasca, and R. T. Batista-Navarro, "Natural language processing for requirements engineering: A systematic mapping study," *ACM Computing Surveys (CSUR)*, vol. 54, no. 3, pp. 1–41, 2021.

[13] A. Aurum and C. Wohlin, *Engineering and managing software requirements*. Springer, 2005, vol. 1.

[14] M. Luisa, F. Mariangela, and N. I. Pierluigi, "Market research for requirements analysis using linguistic tools," *Requirements Engineering*, vol. 9, pp. 40–56, 2004.

[15] M. Kassab, C. Neill, and P. Laplante, "State of practice in requirements engineering: contemporary data," *Innovations in Systems and Software Engineering*, vol. 10, no. 4, pp. 235–241, 2014.

[16] E. Kamsties and B. Peach, "Taming ambiguity in natural language requirements," in *Proceedings of the Thirteenth international conference on Software and Systems Engineering and Applications*, vol. 1315, 2000.

[17] F. de Bruijn and H. L. Dekkers, "Ambiguity in natural language software requirements: A case study," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16*. Springer, 2010, pp. 233–247.

[18] M. Aref *et al.*, "Detecting and resolving ambiguity approach in requirement specification: implementation, results and evaluation," *International Journal of Intelligent Computing and Information Sciences*, vol. 18, no. 1, pp. 27–36, 2018.

[19] G. Lami, S. Gnesi, F. Fabbrini, M. Fusani, and G. Trentanni, "An automatic tool for the analysis of natural language requirements," *Informe técnico, CNR Information Science and Technology Institute, Pisa, Italia, Setiembre*, 2004.

[20] F. Chantree, B. Nuseibeh, A. De Roeck, and A. Willis, "Identifying nocuous ambiguities in natural language requirements," in *14th IEEE International Requirements Engineering Conference (RE'06)*. IEEE, 2006, pp. 59–68.

[21] S. Ezzini, S. Abualhaija, C. Arora, M. Sabetzadeh, and L. Briand, "Maana: An automated tool for domain-specific handling of ambiguity," in *2021*

IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). IEEE, 2021, pp. 188–189.

[22] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "TAPHSIR: towards anaphoric ambiguity detection and ReSolution in requirements," in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 1677–1681.

[23] ——, "Automated handling of anaphoric ambiguity in requirements: a multi-solution study," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 187–199.

[24] R. Izhar, S. N. Bhatti, and S. A. Alharthi, "Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements," *IEEE Access*, 2025.

[25] I. M. Subedi, M. Singh, V. Ramasamy, and G. S. Walia, "Application of back-translation: a transfer learning approach to identify ambiguous software requirements," in *Proceedings of the 2021 ACM Southeast Conference*, 2021, pp. 130–137.

[26] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy, and A. Ibrahim, "Score-based automatic detection and resolution of syntactic ambiguity in natural language requirements," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 651–661.

[27] G. Kamath, S. Schuster, S. Vajjala, and S. Reddy, "Scope ambiguities in large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 738–754, 2024.

[28] S. F. Tjong and D. M. Berry, "The design of SREE—a prototype potential ambiguity finder for requirements specifications and lessons learned," in *International Working Conference on Requirements Engineering: Foundation for Software Quality*. Springer, 2013, pp. 80–95.

[29] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[30] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[31] P. Oghabi and N. Kahani, "Multi-label ambiguity detection in software requirements using language models," in *35th IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*, 2025.

[32] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16.* Springer, 2010, pp. 218–232.

[33] A. Ferrari, G. O. Spagnolo, and S. Gnesi, "Pure: A dataset of public requirements documents," in *2017 IEEE 25th international requirements engineering conference (RE).* IEEE, 2017, pp. 502–505.

[34] A. Ferrari, G. Lipari, S. Gnesi, and G. O. Spagnolo, "Pragmatic ambiguity detection in natural language requirements," in *2014 IEEE 1st International Workshop on Artificial Intelligence for Requirements Engineering (AIRE).* IEEE, 2014, pp. 1–8.

[35] K. Aslam, F. Iqbal, A. Altaf, N. Hussain, M. G. Villar, E. S. Flores, I. D. L. T. Diez, and I. Ashraf, "Detecting pragmatic ambiguity in requirement specification using novel concept maximum matching approach based on graph network," *IEEE Access*, vol. 12, pp. 15 651–15 661, 2024.

[36] A. Ferrari and S. Gnesi, "Using collective intelligence to detect pragmatic ambiguities," in *2012 20th IEEE International Requirements Engineering Conference (RE).* IEEE, 2012, pp. 191–200.

[37] F. Zait and N. Zarour, "Addressing lexical and semantic ambiguity in natural language requirements," in *2018 Fifth International Symposium on Innovation in Information and Communication Technology (ISIICT).* IEEE, 2018, pp. 1–7.

[38] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, "A survey of large language models," *arXiv preprint arXiv:2303.18223*, 2023.

[39] S. F. Chen and J. Goodman, "An empirical study of smoothing techniques for language modeling," *Computer Speech & Language*, vol. 13, no. 4, pp. 359–394, 1999.

[40] T. Mikolov, M. Karafiát, L. Burget, J. Cernockỳ, and S. Khudanpur, "Recurrent neural network based language model." in *Interspeech*, vol. 2, no. 3. Makuhari, 2010, pp. 1045–1048.

[41] M. Sundermeyer, R. Schlüter, and H. Ney, "Lstm neural networks for language modeling." in *Interspeech*, vol. 2012, 2012, pp. 194–197.

[42] C. Zhou, Q. Li, C. Li, J. Yu, Y. Liu, G. Wang, K. Zhang, C. Ji, Q. Yan, L. He *et al.*, "A comprehensive survey on pretrained foundation models: A

history from bert to chatgpt," *International Journal of Machine Learning and Cybernetics*, pp. 1–65, 2024.

[43] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension," *arXiv preprint arXiv:1910.13461*, 2019.

[44] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.

[45] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of big data*, vol. 6, no. 1, pp. 1–48, 2019.

[46] H. A. Rowley, S. Baluja, and T. Kanade, "Neural network-based face detection," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 20, no. 1, pp. 23–38, 1998.

[47] J. Chen, D. Tam, C. Raffel, M. Bansal, and D. Yang, "An empirical survey of data augmentation for limited data learning in nlp," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 191–211, 2023.

[48] B. Li, Y. Hou, and W. Che, "Data augmentation approaches in natural language processing: A survey," *Ai Open*, vol. 3, pp. 71–90, 2022.

[49] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

[50] S. Banerjee and A. Lavie, "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments," in *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, 2005, pp. 65–72.

[51] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and psychological measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[52] J. L. Fleiss, "Measuring nominal scale agreement among many raters." *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.

[53] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *biometrics*, pp. 159–174, 1977.

[54] A. O. J. Sabriye and W. M. N. W. Zainon, "A framework for detecting ambiguity in software requirement specification," in *2017 8th International Conference on Information Technology (ICIT)*. IEEE, 2017, pp. 209–213.

[55] C. D. Manning, M. Surdeanu, J. Bauer, J. R. Finkel, S. Bethard, and D. McClosky, "The stanford CoreNLP natural language processing toolkit," in *Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations*, 2014, pp. 55–60.

[56] S. Ezzini, S. Abualhaija, C. Arora, M. Sabetzadeh, and L. C. Briand, "Using domain-specific corpora for improved handling of ambiguity in requirements," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1485–1497.

[57] A. Intana, K. Laosen, P. Nuanchan, N. Pattanakit, and S. Dermchai, "An NLP-based approach for detecting ambiguity of thai software requirements specification," in *2024 21st International Joint Conference on Computer Science and Software Engineering (JCSSE)*. IEEE, 2024, pp. 99–106.

[58] H. Yang, A. De Roeck, V. Gervasi, A. Willis, and B. Nuseibeh, "Analysing anaphoric ambiguity in natural language requirements," *Requirements engineering*, vol. 16, no. 3, pp. 163–189, 2011.

[59] S. Alharbi, "Ambiguity detection in requirements classification task using fine-tuned transformation technique," in *CS & IT Conference Proceedings*, vol. 12, no. 21. CS & IT Conference Proceedings, 2022.

[60] M. H. Osman and M. F. Zaharin, "Ambiguous software requirement specification detection: An automated approach," in *Proceedings of the 5th International Workshop on Requirements Engineering and Testing*, 2018, pp. 33–40.

[61] K. Oo, A. Nordin, A. R. Ismail, and S. Sulaiman, "An analysis of ambiguity detection techniques for software requirements specification (SRS)," *International Journal of Engineering & Technology*, vol. 7, p. 501, 05 2018.

[62] Z. Zhang and L. Ma, "Using machine learning for automated detection of ambiguity in building requirements," in *EC3 Conference 2023*, vol. 4. European Council on Computing in Construction, 2023, pp. 0–0.

[63] C. D. A. M. Vieira, "Classification of requirements ambiguity through machine learning techniques," Bachelor's thesis, Pontifícia Universidade Católica de Minas Gerais, Belo Horizonte, Brazil, 2022.

[64] K. H. Oo, "Comparing accuracy between svm, random forest, k-nn text classifier algorithms for detecting syntactic ambiguity in software require-

ments," in *International Conference on Information Systems and Intelligent Applications*. Springer, 2022, pp. 43–58.

[65] H. Yang, A. Willis, A. De Roeck, and B. Nuseibeh, "Automatic detection of nocuous coordination ambiguities in natural language requirements," in *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*, 2010, pp. 53–62.

[66] T. Mahbub, D. Dghaym, A. Shankarnarayanan, T. Syed, S. Shapsough, and I. Zualkernan, "Can GPT-4 aid in detecting ambiguities, inconsistencies, and incompleteness in requirements analysis? a comprehensive case study." *IEEE Access*, 2024.

[67] M. Honnibal, I. Montani, S. Van Landeghem, A. Boyd *et al.*, "spaCy: Industrial-strength natural language processing in python," 2020.

[68] A. Ferrari, G. Gori, B. Rosadini, I. Trotta, S. Bacherini, A. Fantechi, and S. Gnesi, "Detecting requirements defects with NLP patterns: an industrial experience in the railway domain," *Empirical Software Engineering*, vol. 23, no. 6, pp. 3684–3733, 2018.

[69] B. Gleich, O. Creighton, and L. Kof, "Ambiguity detection: Towards a tool explaining ambiguity sources," in *Requirements Engineering: Foundation for Software Quality: 16th International Working Conference, REFSQ 2010, Essen, Germany, June 30–July 2, 2010. Proceedings 16*. Springer, 2010, pp. 218–232.

[70] M. Zakeri-Nasrabadi and S. Parsa, "Natural language requirements testability measurement based on requirement smells," *Neural Computing and Applications*, vol. 36, no. 21, pp. 13 051–13 085, 2024.

[71] P. Szymański and T. Kajdanowicz, "A scikit-based Python environment for performing multi-label classification," *ArXiv e-prints*, Feb. 2017.

[72] M. R. Costa-Jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard *et al.*, "No language left behind: Scaling human-centered machine translation," *arXiv preprint arXiv:2207.04672*, 2022.

[73] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[74] Hugging Face, "Evaluate: A library for easily evaluating machine learning models and datasets," 2022, accessed: July 27, 2025. [Online]. Available: https://github.com/huggingface/evaluate

[75] Google LLC, "Understanding the BLEU score," https://cloud.google.com/translate/automl/docs/evaluate, 2020, accessed: July 27, 2025.

[76] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[77] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[78] A. F. Adoma, N.-M. Henry, and W. Chen, "Comparative analyses of bert, roberta, distilbert, and xlnet for text-based emotion recognition," in *2020 17th international computer conference on wavelet active media technology and information processing (ICCWAMTIP)*. IEEE, 2020, pp. 117–121.

[79] C. Y. Sy, L. L. Maceda, M. J. P. Canon, and N. M. Flores, "Beyond BERT: Exploring the efficacy of RoBERTa and ALBERT in supervised multiclass text classification." *International Journal of Advanced Computer Science & Applications*, vol. 15, no. 3, 2024.

[80] A. Sbei, K. ElBedoui, and W. Barhoumi, "Assessing the efficiency of transformer models with varying sizes for text classification: A study of Rule-Based annotation with DistilBERT and other transformers," *Vietnam Journal of Computer Science*, vol. 1, p. 28, 2024.

[81] Hugging Face, "RoBERTa model documentation," https://huggingface.co/transformers/v3.0.2/model_doc/roberta.html, 2020, accessed: July 27, 2025.

[82] ——, "DistilBERT model documentation," https://huggingface.co/transformers/v3.0.2/model_doc/distilbert.html, 2020, accessed: July 27, 2025.

[83] ——, "Trainer," https://huggingface.co/docs/transformers/main_classes/trainer, 2020, accessed: July 27, 2025.

[84] K. Kaur and P. Kaur, "MNoR-BERT: multi-label classification of non-functional requirements using bert," *Neural Computing and Applications*, vol. 35, no. 30, pp. 22 487–22 509, 2023.

[85] A. Berger and S. Guda, "Threshold optimization for f measure of macro-averaged precision and recall," *Pattern Recognition*, vol. 102, p. 107250, 2020.

[86] J. Opitz and S. Burst, "Macro f1 and macro f1," *arXiv preprint arXiv:1911.03347*, 2019.

[87] L. Tang, S. Rajan, and V. K. Narayanan, "Large scale multi-label classification via metalabeler," in *Proceedings of the 18th international conference on World wide web*, 2009, pp. 211–220.

[88] A. Doknic and T. Möller, "MLMC: Interactive multi-label multi-classifier evaluation without confusion matrices," *arXiv preprint arXiv:2501.14460*, 2025.

[89] P. Dhal and C. Azad, "A multi-stage multi-objective GWO based feature selection approach for multi-label text classification," in *2022 2nd International Conference on Intelligent Technologies (CONIT)*. IEEE, 2022, pp. 1–5.

[90] M. Heydarian, T. E. Doyle, and R. Samavi, "MLCM: Multi-label confusion matrix," *Ieee Access*, vol. 10, pp. 19 083–19 095, 2022.

[91] P. Domingos, "A few useful things to know about machine learning," *Communications of the ACM*, vol. 55, no. 10, pp. 78–87, 2012.

Appendix A

Additional Figures

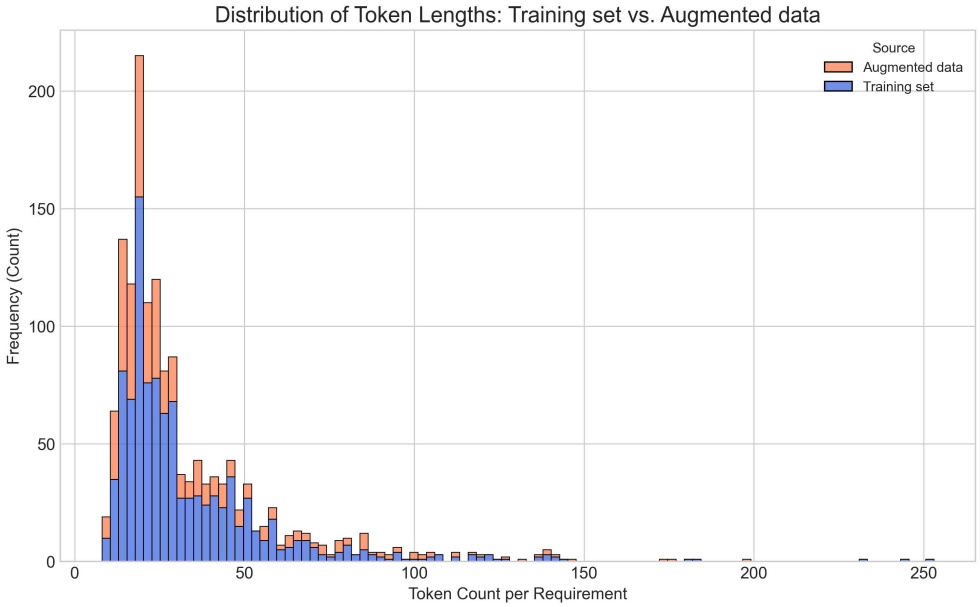


Figure A.1: Distribution of Token Lengths in DistilBertTokenizer

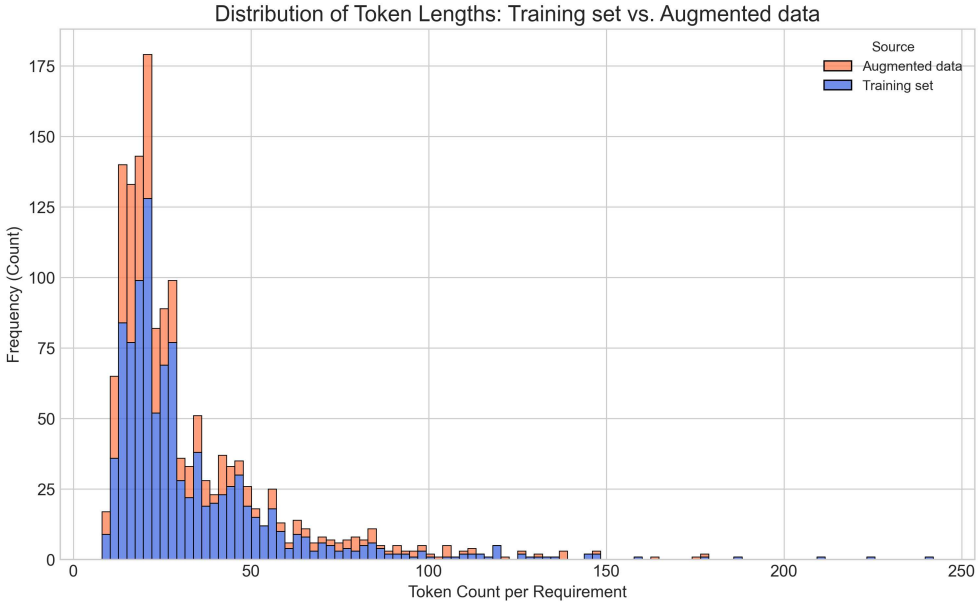


Figure A.2: Distribution of Token Lengths in RobertaTokenizer